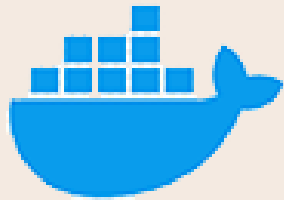


Docker&Kubernetes

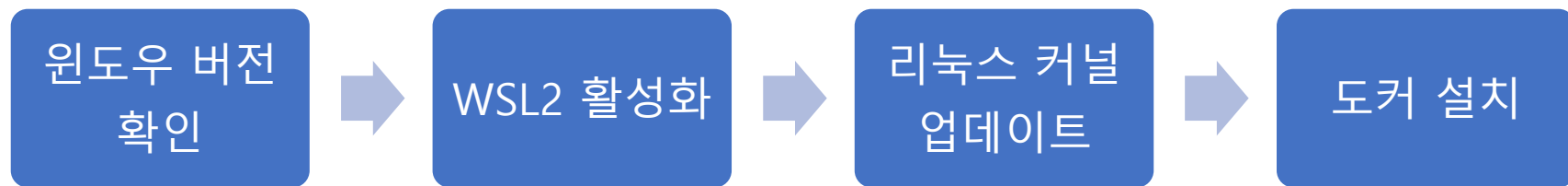
도커 설치하기



☰ 작업하고자 하는 내용

Window 용 Docker Desktop를 설치하기 위해서는 다음과 같은 과정으로 진행하면 됩니다.

구현할 내용



☰ 사전 준비

step00 : 윈도우를 최신 버전으로 업데이트 합니다.

윈도우를 업데이트 한 다음 `1903/1909 이후` 버전인지 확인합니다.

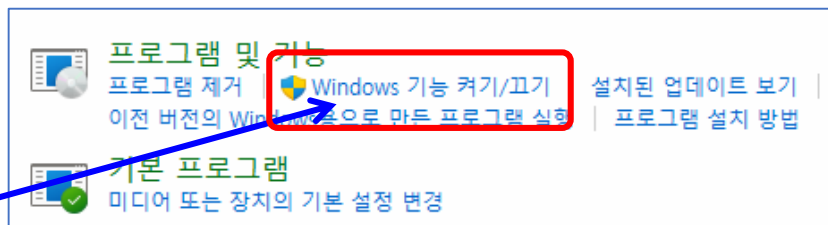
Windows 기능 켜기/끄기 창

step01 : Windows 기능 켜기/끄기

윈도우 시작 메뉴에서 [제어판]-[프로그램]-[프로그램 및 기능]-[Windows 기능 켜기/끄기] 기능을 클릭합니다.

버전 Window 11

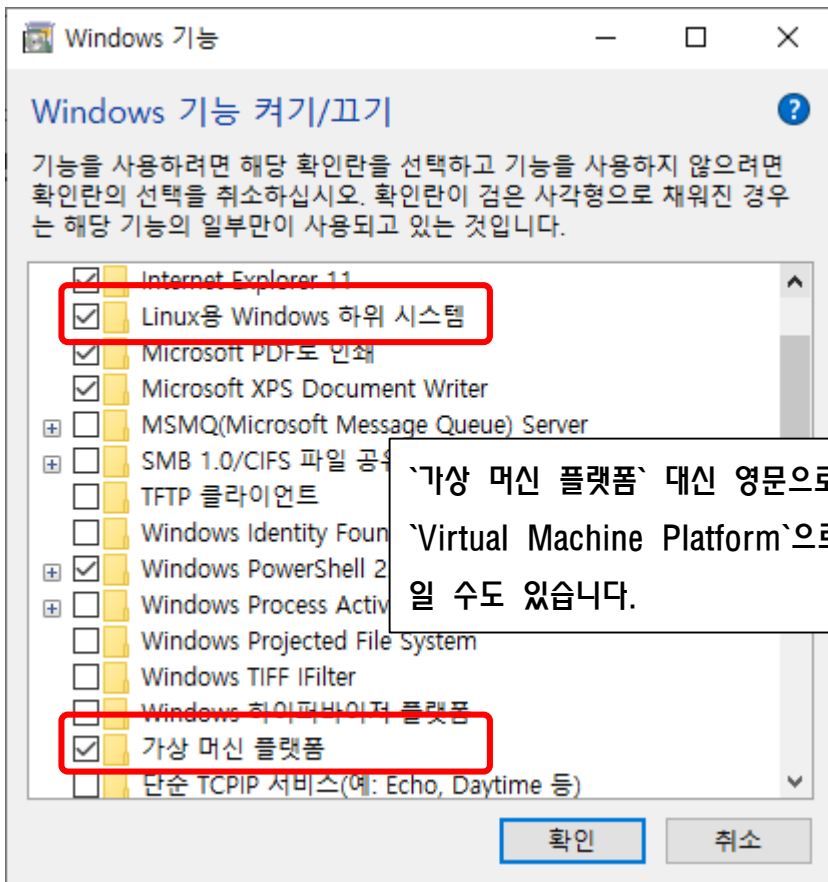
컴퓨터 설정 변경



☰ Windows 기능 켜기/끄기 창

step02 : 기능 활성화

[Linux용 Windows 하위 시스템]과 [가상 머신 플랫폼] 기능을 **체크 on 상태**로 변경한 다음 확인 버튼을 클릭합니다.



요구 사항

WSL(Windows Subsystem for Linux)은 Windows에서 직접 Linux 환경을 실행할 수 있도록 하는 기능입니다.

이를 통해 별도의 가상 머신(VM)이나 Dual Booting이 없이도 Windows에서 Linux 명령어, 도구, 애플리케이션을 사용할 수 있습니다.

Windows와 Linux 간의 파일 시스템 및 네트워크 공유 가능
다양한 Linux 배포판(Ubuntu, Debian 등) 설치 및 실행 가능
GUI 애플리케이션(Wayland 지원) 실행 가능(WSL 2)
Docker 등의 개발 환경에서 사용 가능

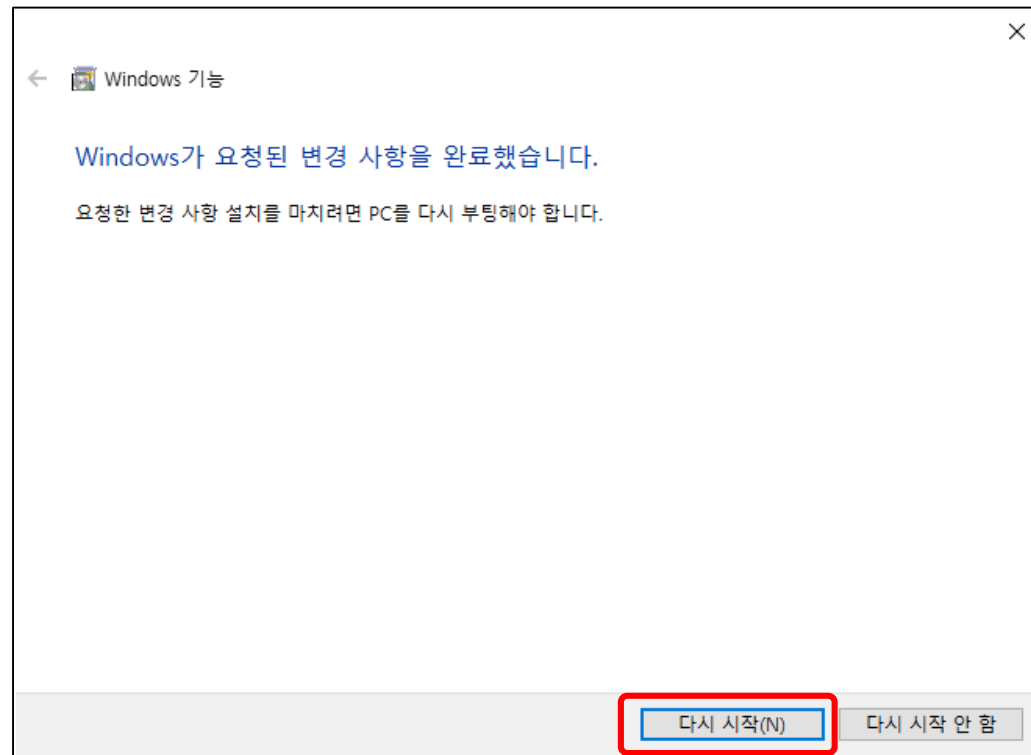
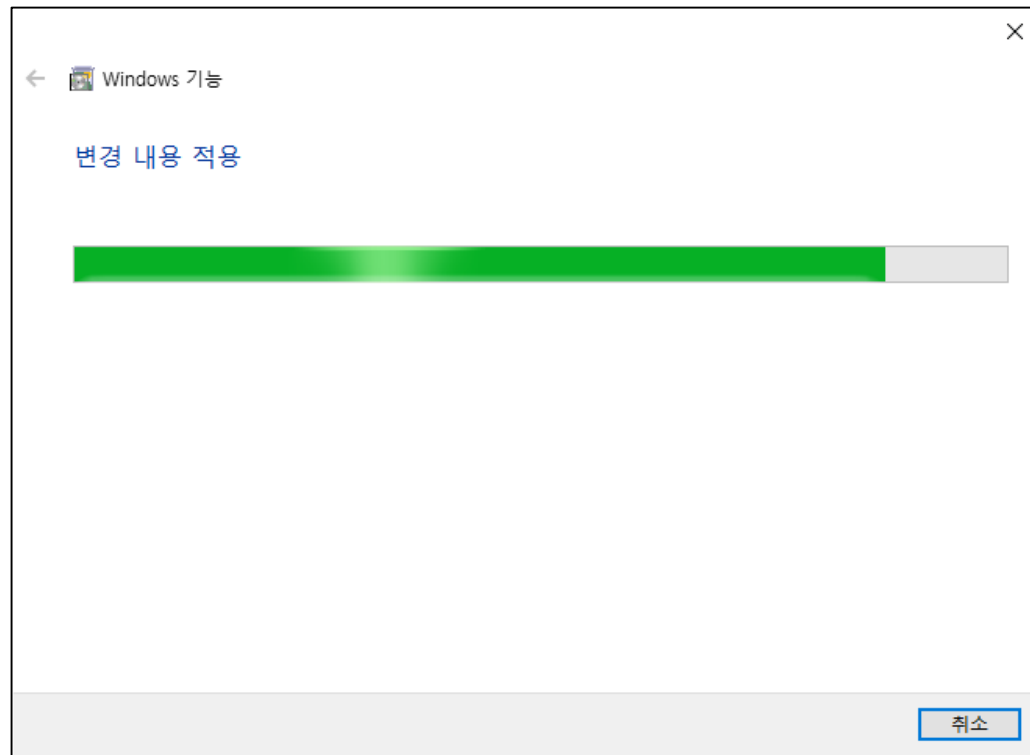
가상 머신 플랫폼(Virtual Machine Platform)은 Windows에서 가상 머신(VM)을 실행할 수 있도록 지원하는 기능입니다.

WSL2나 Windows Sandbox, Hyper-V 등의 기능을 사용하기 위해 필요할 수 있습니다.

☰ Windows 기능 켜기/끄기 창

step03 : 컴퓨터 재부팅

체크한 항목들을 활성화시키기 위하여 컴퓨터를 재시작 하도록 합니다.



리눅스 커널 업데이트

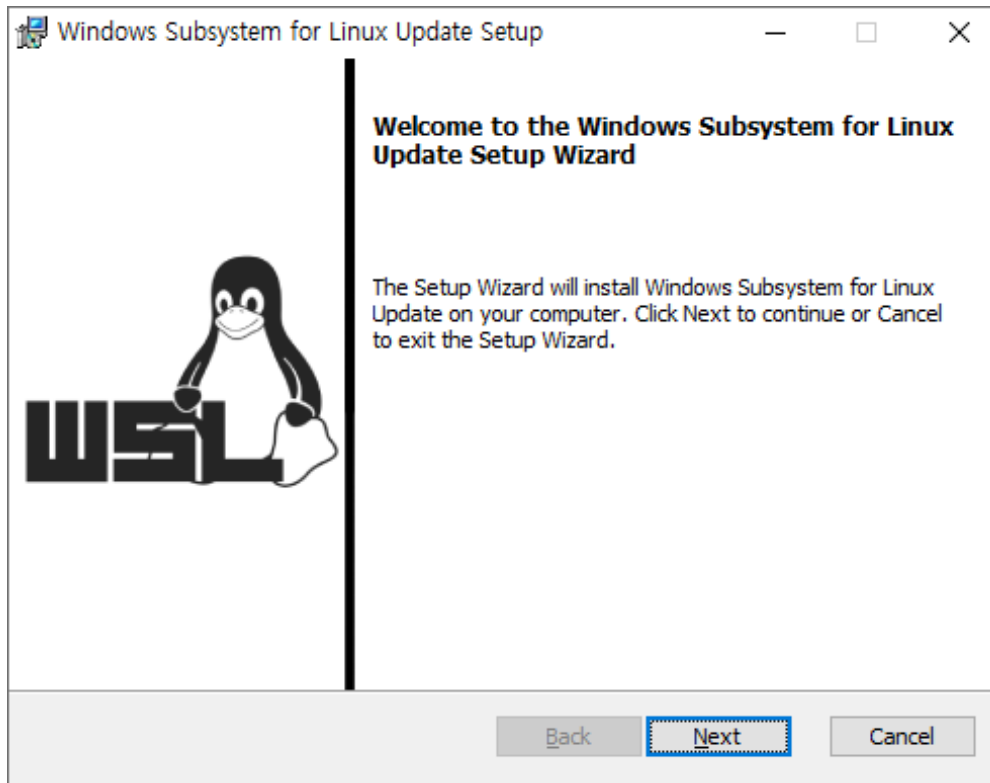
step04 : 웹 브라우저에서 리눅스 커널을 내려 받아서, 커널을 업데이트 합니다.

구글에서 `wsl update download`라고 입력하여 검색을 진행하면 됩니다.

자료 검색이 힘들면 아래의 참조 사이트를 사용하시면 됩니다.

참조 사이트

https://wslstorage.blob.core.windows.net/wslblob/wsl_update_x64.msi

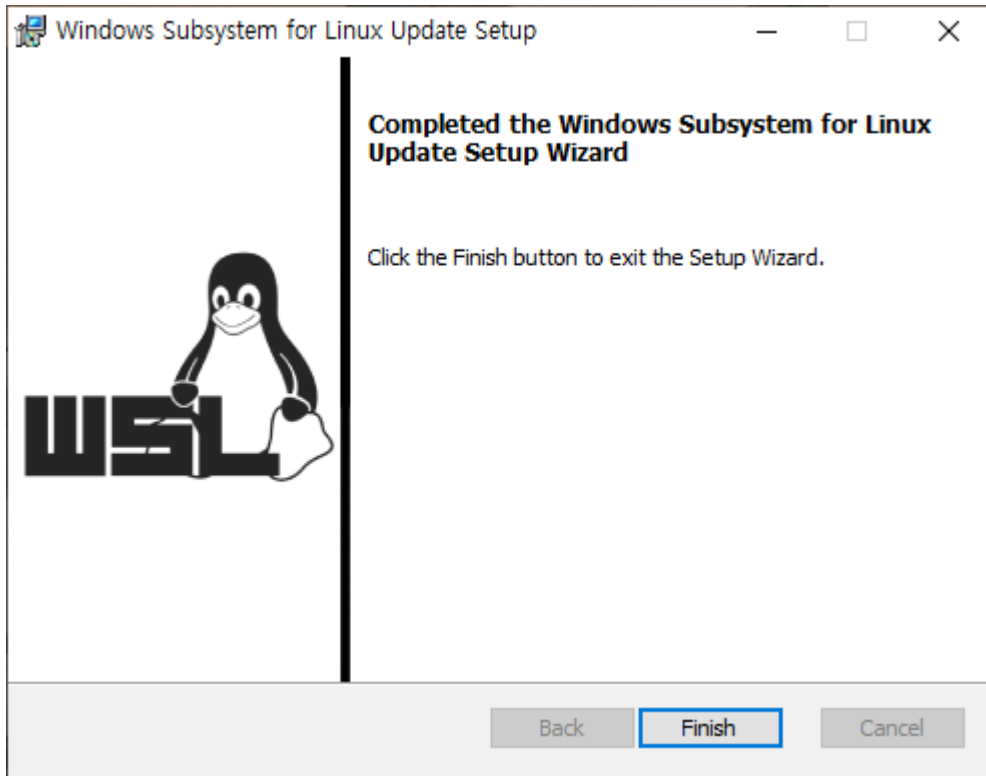


wsl이란?

리눅스용 윈도우 하위 시스템(Windows Subsystem for Linux)이란 Microsoft에서 개발한 Windows에서 리눅스 커널과 GNU 사용자 공간 프로그램들을 Windows 환경에서 직접 실행할 수 있게 해주기 위한 기능입니다.

☰ 리눅스 커널 업데이트

step04 : 웹 브라우저에서 리눅스 커널을 내려 받아서, 커널을 업데이트 합니다.

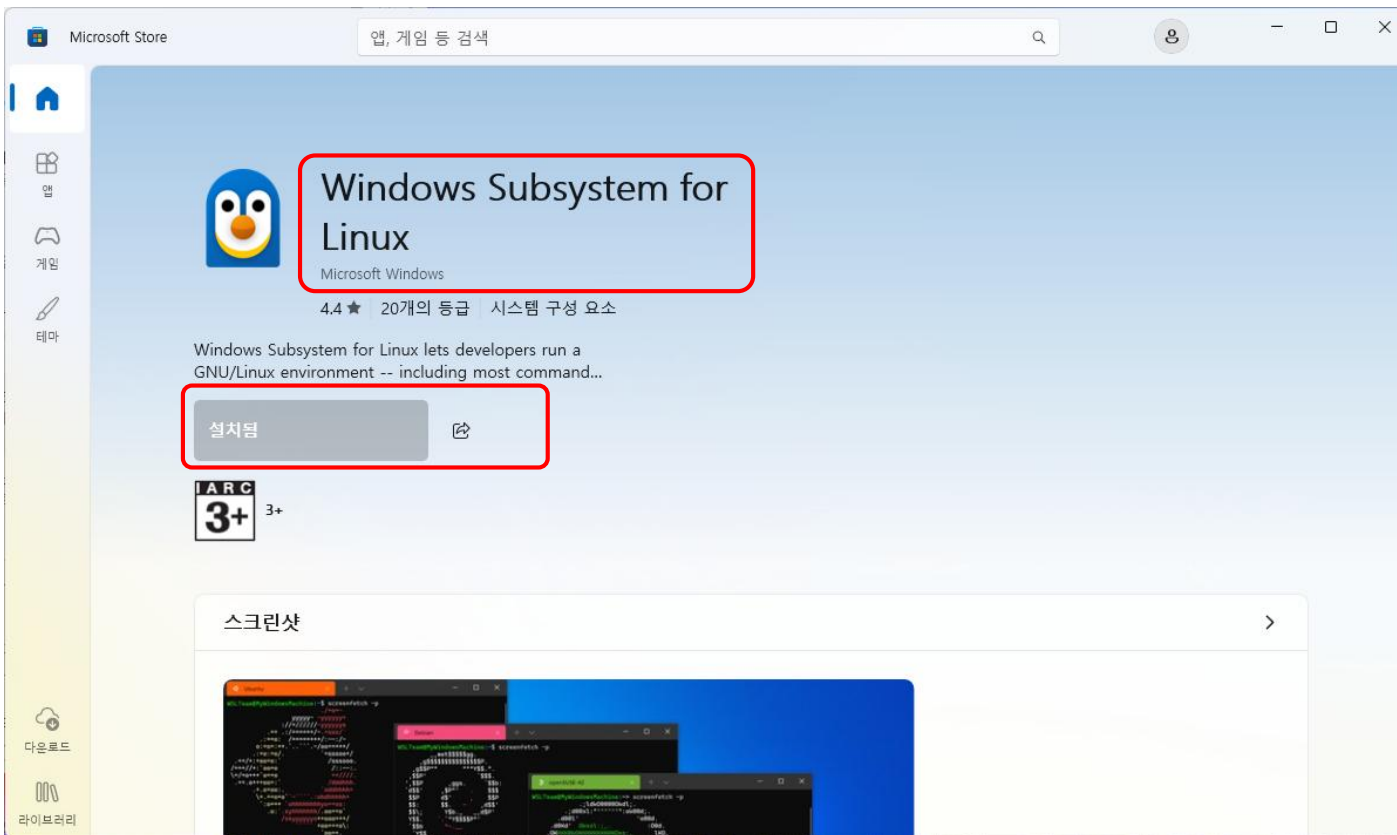


☰ 리눅스 커널 업데이트

step04 : 커널 업데이트가 잘 되지 않는 경우 다음 명령어를 실행하면 해결할 수 있습니다.

해결 방법

start ms-windows-store://pdp/?productid=9P9TQF7MRM4R



☰ Windows용 도커 데스크탑 설치

step05 : 웹 브라우저에서 윈도우용 도커 데스크탑을 다운로드하고, 설치를 진행합니다.

환경 설정 항목에서 체크 항목들을 on 상태로 지정하고 ok 버튼을 클릭합니다.

다소 시간이 걸릴 수 있습니다.

참조 사이트

<https://docs.docker.com/docker-for-windows/install>



Install Docker Desktop on Windows

Copy as Markdown

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR more than \$10 million USD in annual revenue) requires a [paid subscription](#).

This page provides download links, system requirements, and step-by-step installation instructions for Docker Desktop on Windows.

Docker Desktop for Windows - x86_64

Docker Desktop for Windows - x86_64 on the Microsoft Store

Docker Desktop for Windows - Arm (Early Access)

☰ Windows용 도커 데스크탑 설치

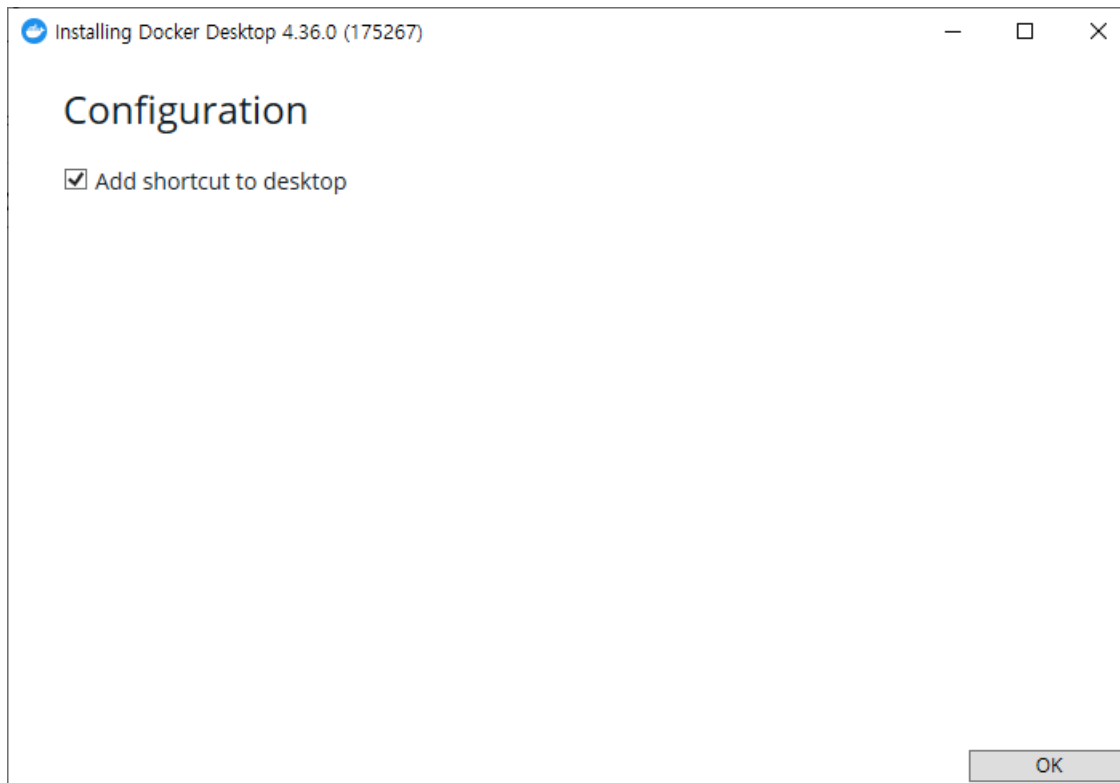
step05 : 웹 브라우저에서 윈도우용 도커 데스크탑을 다운로드하고, 설치를 진행합니다.

환경 설정 항목에서 체크 항목들을 on 상태로 지정하고 ok 버튼을 클릭합니다.

다소 시간이 걸릴 수 있습니다.

참조 사이트

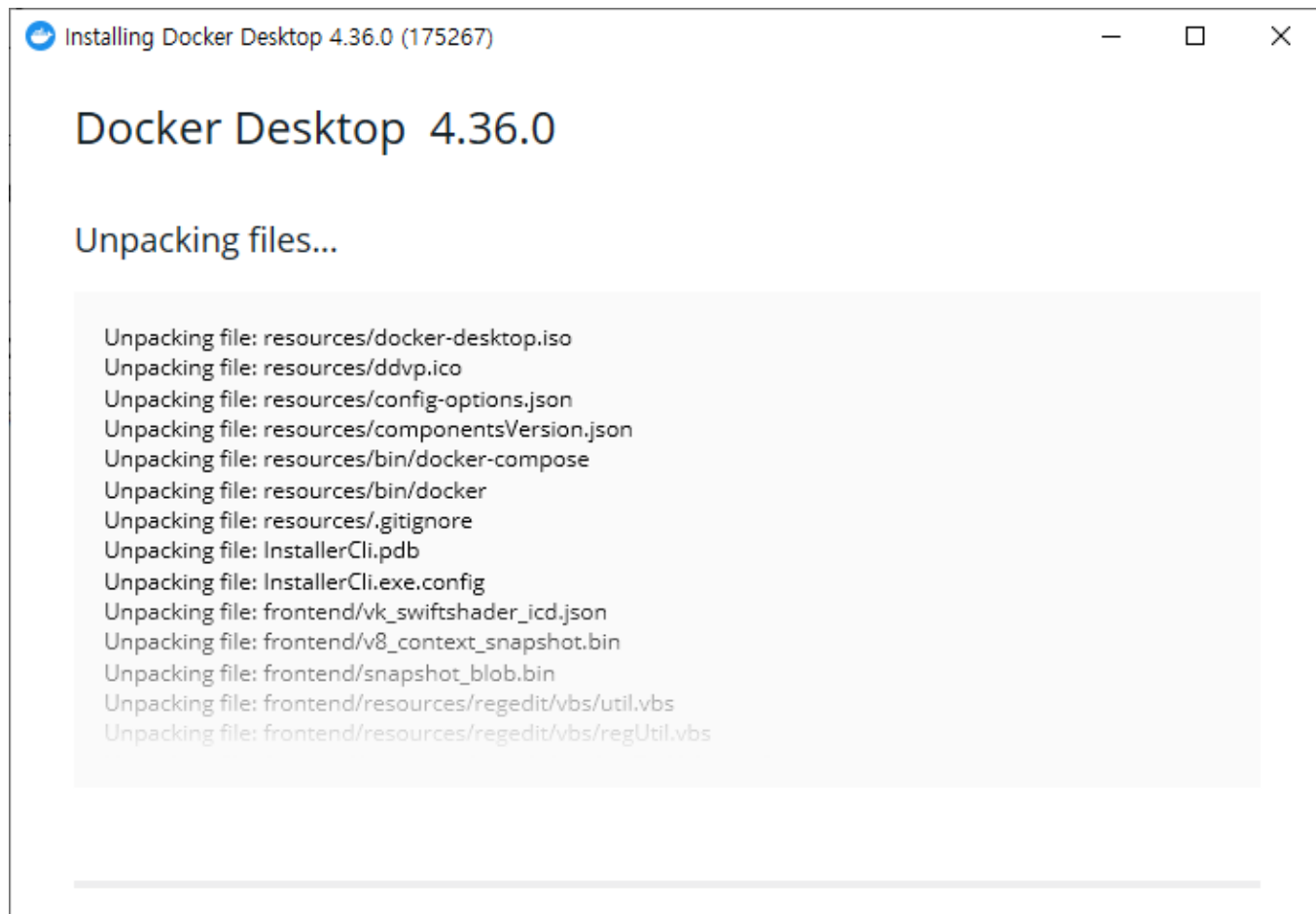
<https://docs.docker.com/docker-for-windows/install>



☰ Windows용 도커 데스크탑 설치

step05 : 웹 브라우저에서 윈도우용 도커 데스크탑을 다운로드하고, 설치를 진행합니다.

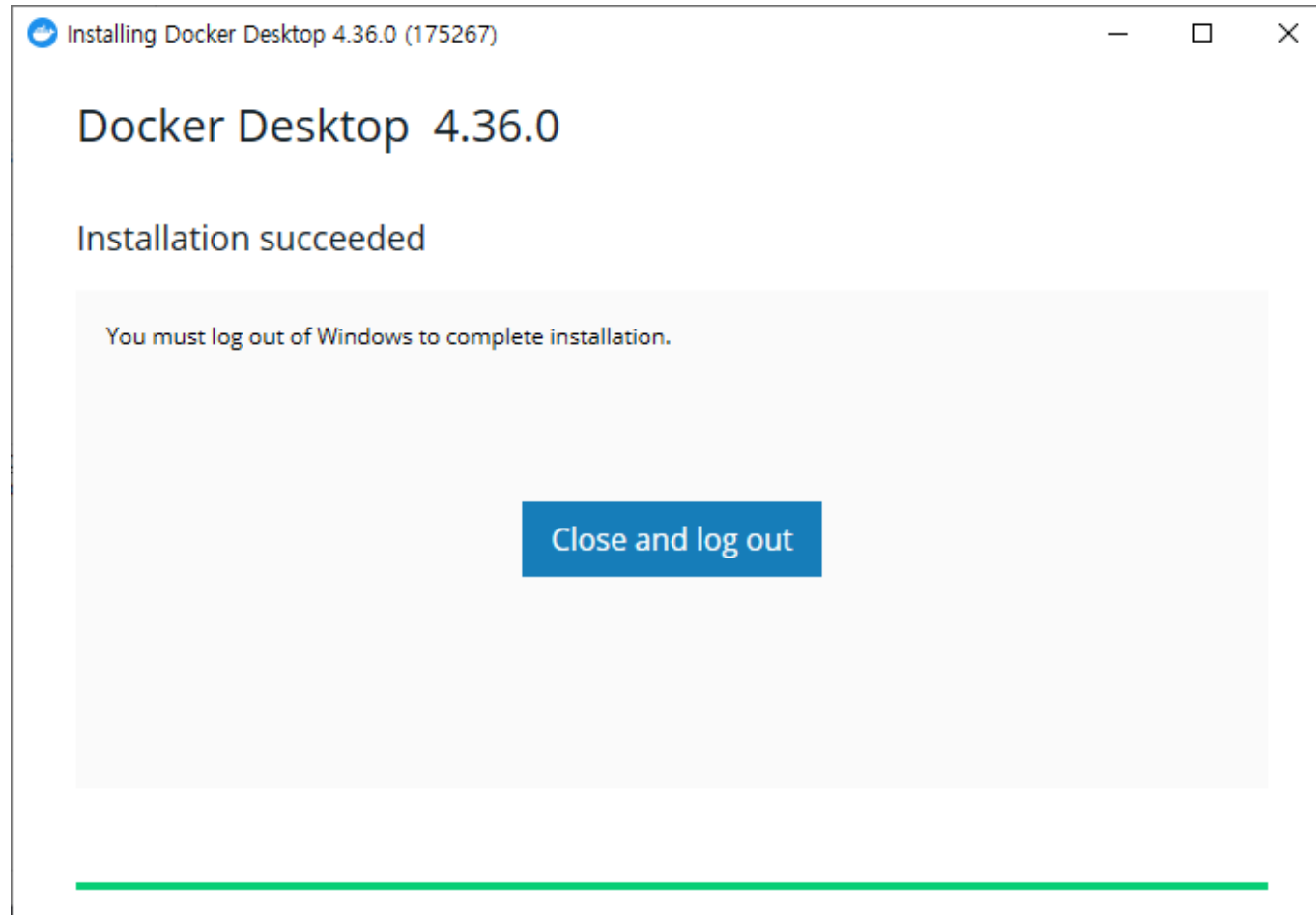
연관된 파일들을 모두 압축 해제하는 화면입니다. 다소 시간이 좀 걸릴수 있습니다.



☰ Windows용 도커 데스크탑 설치

step06 : 설치를 완료하고 나서 [Close and Logout]을 클릭하여 윈도우를 다시 로그인합니다.

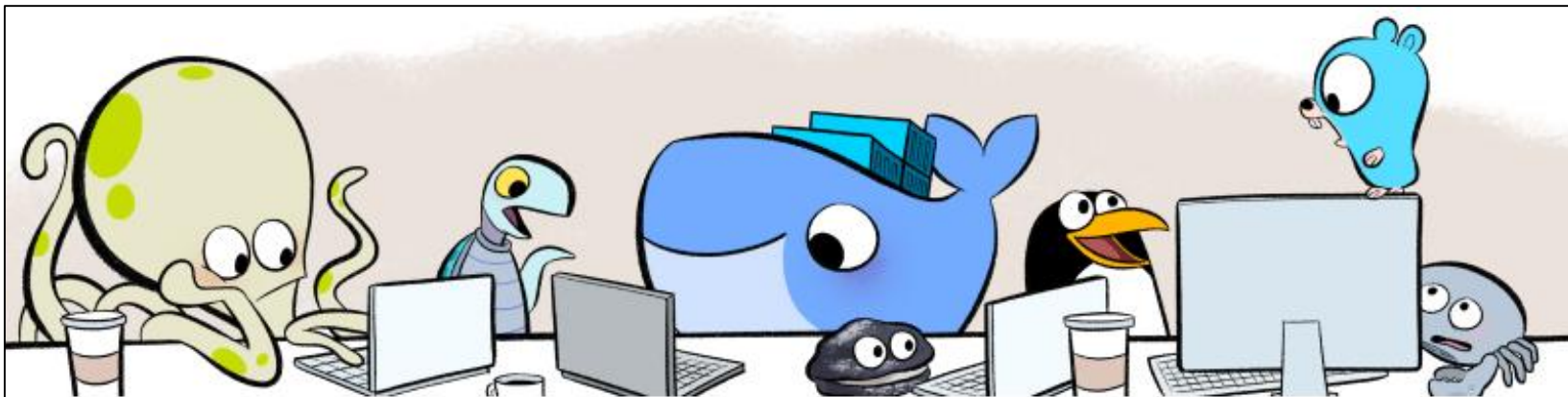
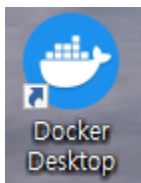
설치가 끝나고 나면 바탕 화면에 `도커 데스크탑`이라는 아이콘이 새로 추가됩니다.



☰ 도커 데스크탑 화면

☰ 도커 실행하기

바탕 화면의 `도커 데스크탑`을 더블 클릭한 다음 `Accept` 버튼을 클릭합니다.



Docker Subscription Service Agreement

By selecting accept, you agree to the [Subscription Service Agreement](#), the [Docker Data Processing Agreement](#), and the [Data Privacy Policy](#).

Commercial use of Docker Desktop at a company of more than 250 employees OR more than \$10 million in annual revenue requires a paid subscription (Pro, Team, or Business). [See subscription details](#)

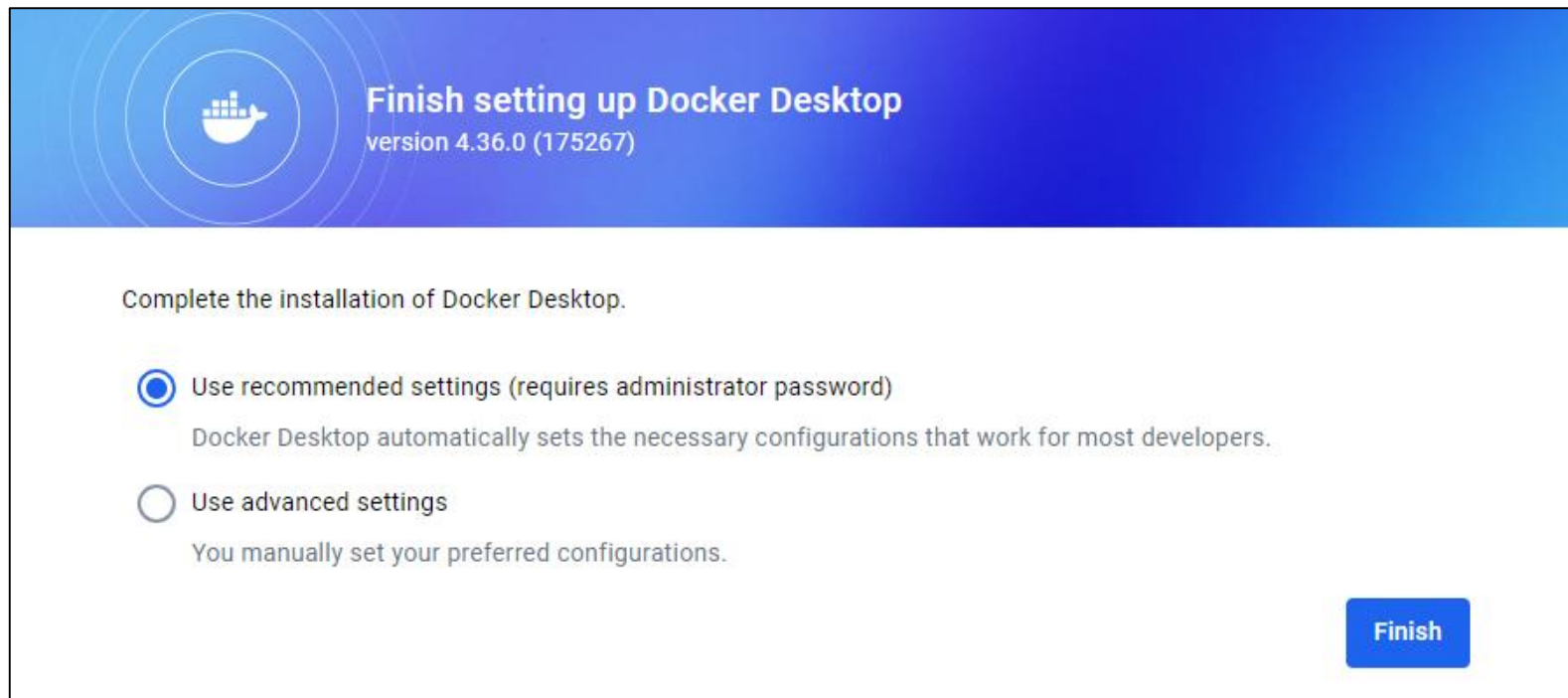
[View Full Terms](#)

Accept

Close

☰ 도커 실행하기

추천하는 기본 셋팅 값을 유지한 상태에서 `Finish` 버튼을 클릭합니다.



☰ 도커 실행하기

웰컴 페이지입니다.

`Skip` 버튼을 클릭하고 다음으로 이동합니다.

Welcome to Docker

[Skip](#)

Work

Personal

Using Docker for work?

We recommend signing in with your work email address.

Continue

Or



Create an account



☰ 도커 실행하기

설문 조사를 위한 화면입니다.

`Skip` 버튼을 클릭하고 다음으로 이동합니다.

Skip



Welcome Survey

Step 1 of 2

What's your role?

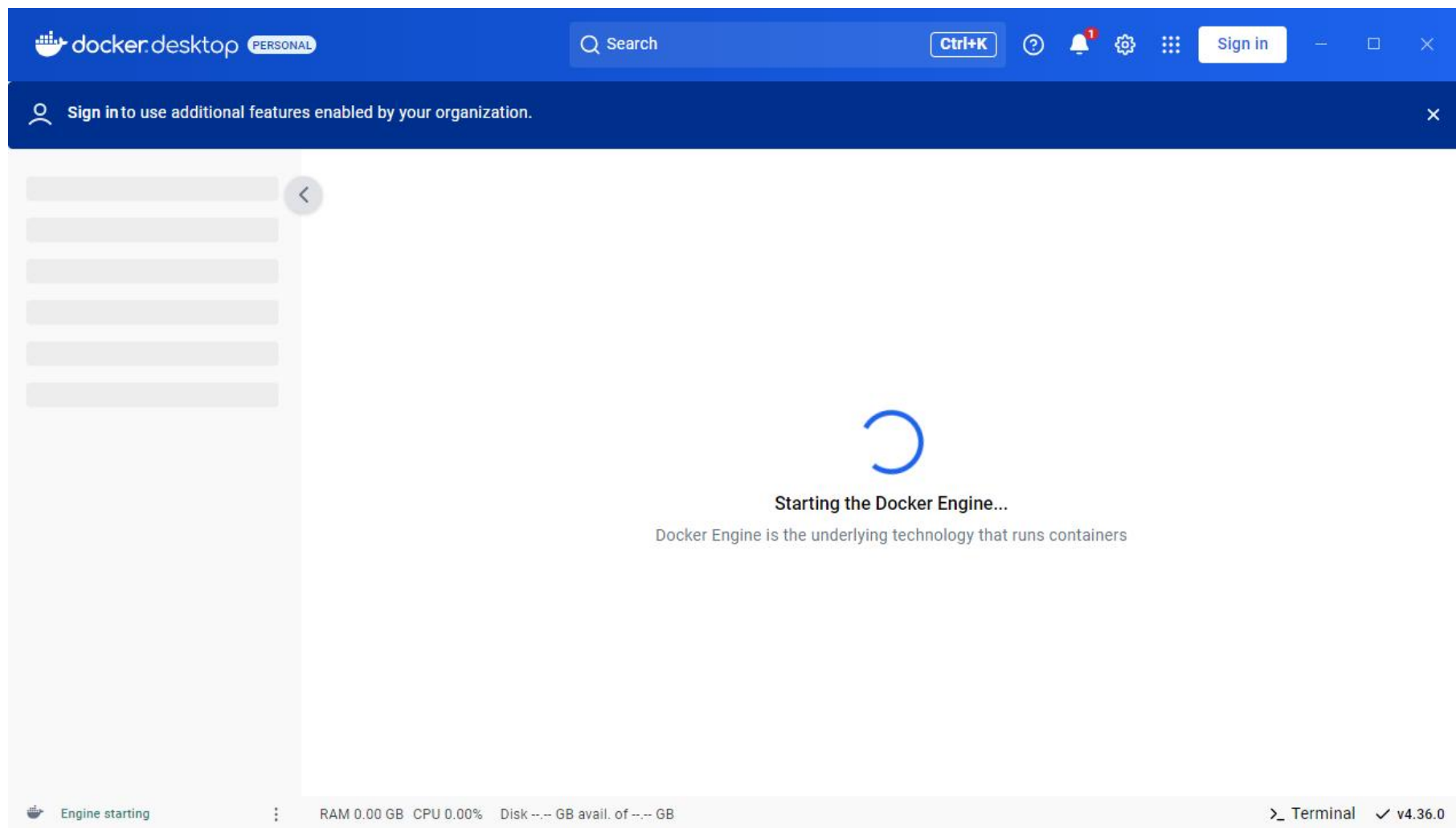
☐ Full-stack developer	☒ Front-end developer
☐ Back-end developer	☐ Site reliability engineer
☐ Platform engineer	☐ DevOps specialist
☐ Infrastructure manager	☐ System administrator
☐ Security engineer	☐ Data scientist
☐ Product manager	☐ Teacher
☐ Student	☐ Other

☰ 도커 데스크탑 화면

☰ 도커 실행하기

도커 데스크탑의 시작 화면입니다.

Docker Engine이 시작되고
있음을 알려 줍니다.



도커 실행하기

도커 데스크탑이 실행되었습니다.

화면의 좌측 하단을 보면
`Engine running`이라고
메시지를 보여 주고
있습니다.

항목	설명
Containers	독립적으로 실행이 가능한 프로그램 묶음을 저장해 놓은 공간
Images	컨테이너를 생성하는 데 필요한 읽기 전용의 템플릿
Volume	컨테이너의 실행 여부와 상관이 없이 항상 데이터 접근이 가능한 별도의 외부 저장소
Builds	컨테이너 이미지 빌드와 관련된 정보를 제공합니다.
Docker Scout	컨테이너 이미지의 보안 및 효율성에 관련된 정보를 분석하고 제공합니다.
Extensions	Docker Desktop의 기능을 확장하거나 사용자 정의 도구를 추가하는 데 사용됩니다.

파일의 위치

C:\Users\사용자컴퓨터이름\AppData\Local\Docker\ws\disk 폴더 아래에 `docker\_data.vhdx`라는 이름으로 저장되어 있습니다.
WSL 2(Windows Subsystem for Linux 2) 기반 Docker가 컨테이너 데이터와 볼륨, 이미지, 파일 시스템 등을 저장하기 위해 사용하는 가상 하드 디스크 (VHDX) 파일입니다.
참고로 AppData 폴더는 숨김 폴더로 되어 있습니다.

Docker Terminal을
실행할 수 있습니다.

>_ Terminal ✓ v4.36.0

도커 터미널

도커 터미널은 docker과 관련된 명령어를 키보드로 입력할 수 있는 창입니다.

화면 우측 하단을 Terminal 항목을 누르면 다음과 같이 보입니다.

버튼 `Enable`을 클릭하면 터미널 창을 확인할 수 있습니다.

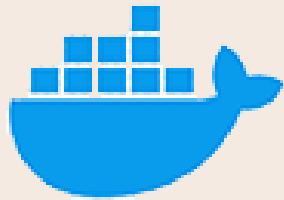
The screenshot shows the Docker Desktop interface. On the left, the 'Containers' section is active. On the right, the 'Terminal' window is open, displaying a Windows PowerShell prompt. The terminal text includes 'Windows PowerShell', 'Copyright (C) Microsoft Corporation. All rights reserved.', and '새로운 크로스 플랫폼 PowerShell 사용 https://aka.ms/pscore6'. Below the terminal window, the 'Enable Docker terminal' button is highlighted with a red circle and the number 1. The terminal window itself has a title bar with a red circle and the number 2. The bottom status bar shows 'RAM 0.84 GB CPU 0.00% Disk 1024.96 GB avail. of 1081.10 GB' and 'v4.36.0'.

명령 프롬프트 단축키

항목	설명
복사	Ctrl + C
붙여 넣기	Ctrl + V
명령 히스토리	커서 key up
화면 지우기	clear

Docker&Kubernetes

도커의 개요



☰ 도커의 개요

☰ 독립된 환경

다음과 같이 시스템에 대하여 살펴 보도록 하겠습니다.

시스템 A



파이썬 2.x



Windows Xp



mysql 5.x

시스템 B



파이썬 3.x



Windows 11



mysql 8.x

시스템 C



파이썬 2.x



Windows 11



mysql 8.x

☰ 도커의 개요

☰ 밀키트와 도커

도커 이미지는 된장 찌개 밀키트 상품과 유사한 개념입니다.



Meal Kit Analogy

1. Meal Kit Product

The Meal Kit contains all the ingredients and instructions to make Doenjang Jjigae.



What's Inside?

- Doenjang Sauce
- Tofu
- Zucchini
- Mushroom
- Potato
- Green Onion
- Instruction Manual

2. Customers Cook the Meal Kit

Each customer cooks the same meal kit in their own way and environment.



The Meal Kit is the same, but each customer creates their own dish.
Same ingredients, different results based on the cook.

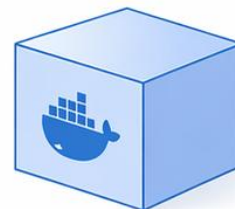
vs

Docker Analogy



1. Docker Image

The Docker Image contains everything needed to run an application.



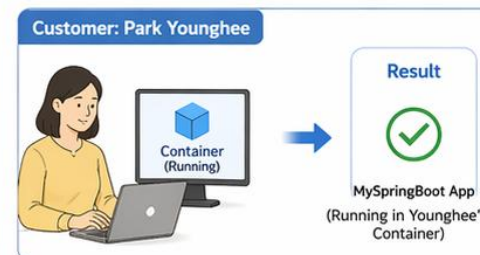
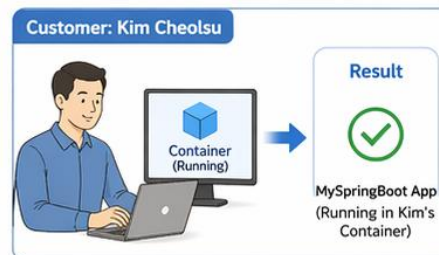
MySpringBoot Image

What's Inside?

- Application Code
- Libraries & Dependencies
- Runtime (JRE)
- Configuration
- Environment
- Startup Command

2. Customers Run Containers from the Image

Each customer runs a container from the same image, in their own environment.



The Image is the same, but each customer runs their own container.
Same image, different running instances (isolated environments).

☰ 도커의 개요

☰ 자바 클래스와 도커

도커 이미지는 자바 클래스로부터 만들어진 객체와 유사한 개념입니다.



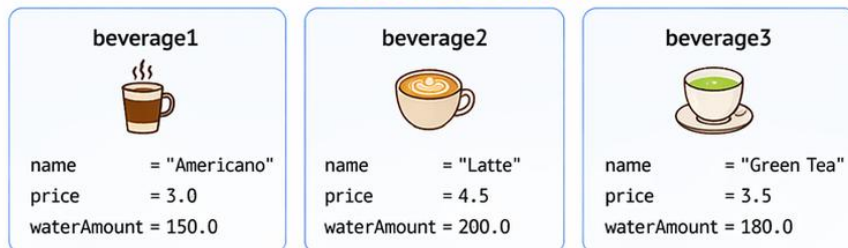
Java OOP Analogy

Class vs Object

CLASS (Blueprint)

```
public class Beverage {  
    private String name ;           // beverage name  
    private double price ;          // unit price  
    private double waterAmount ;    // water amount  
}
```

OBJECTS (Instances)



Class = Blueprint

Object = Instance created from the class



Docker Analogy

Image vs Container

DOCKER IMAGE (Blueprint)

```
FROM openjdk:17-jdk-slim  
COPY app.jar /app/app.jar  
EXPOSE 8080  
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```



MySpringBoot Image

CONTAINERS (Instances)

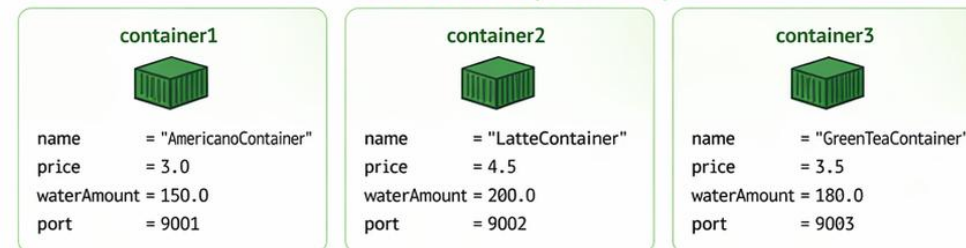


Image = Blueprint (template)

Container = Instance created from the image (runtime object)

Analogy

Both are blueprints.
Instances are created from
the blueprint and run independently.

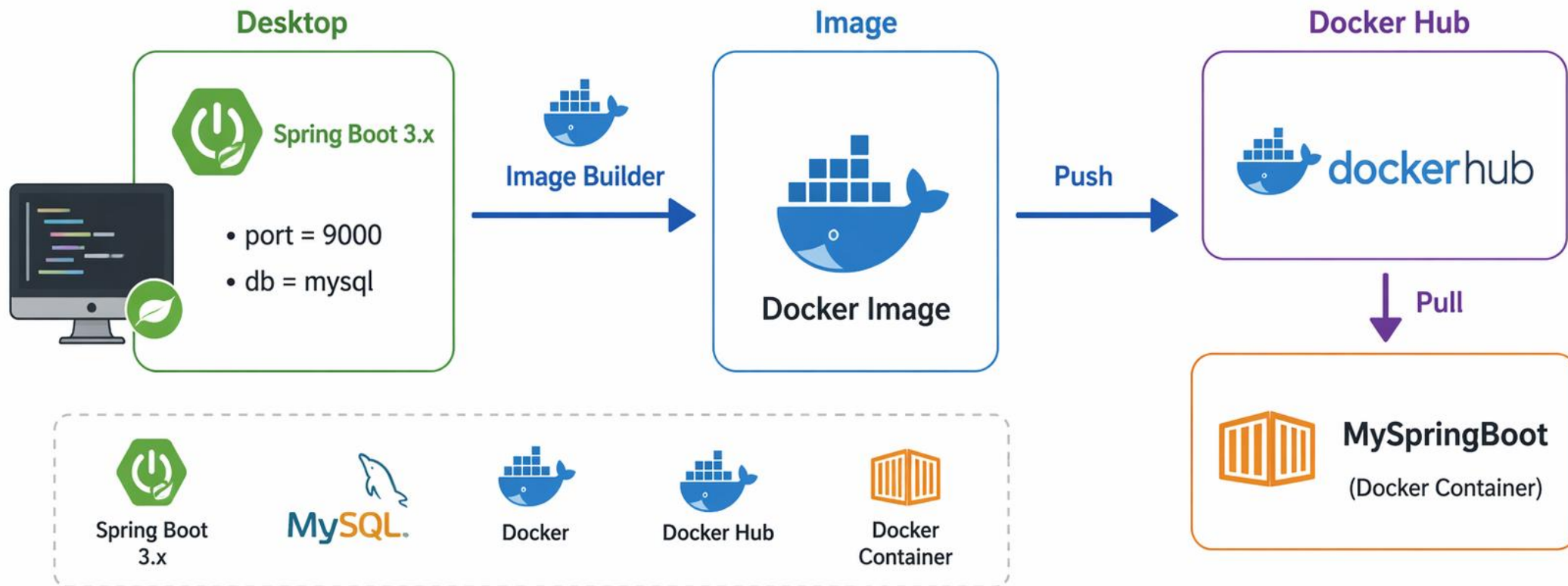
☰ 도커의 개요

☰ 이미지 생성

여러 대의 컴퓨터에 동일한 셋팅이 되어 있는 스프링 부트 애플리케이션을 설정한다고 가정해 봅니다.



Spring Boot 3.x Deployment Flow

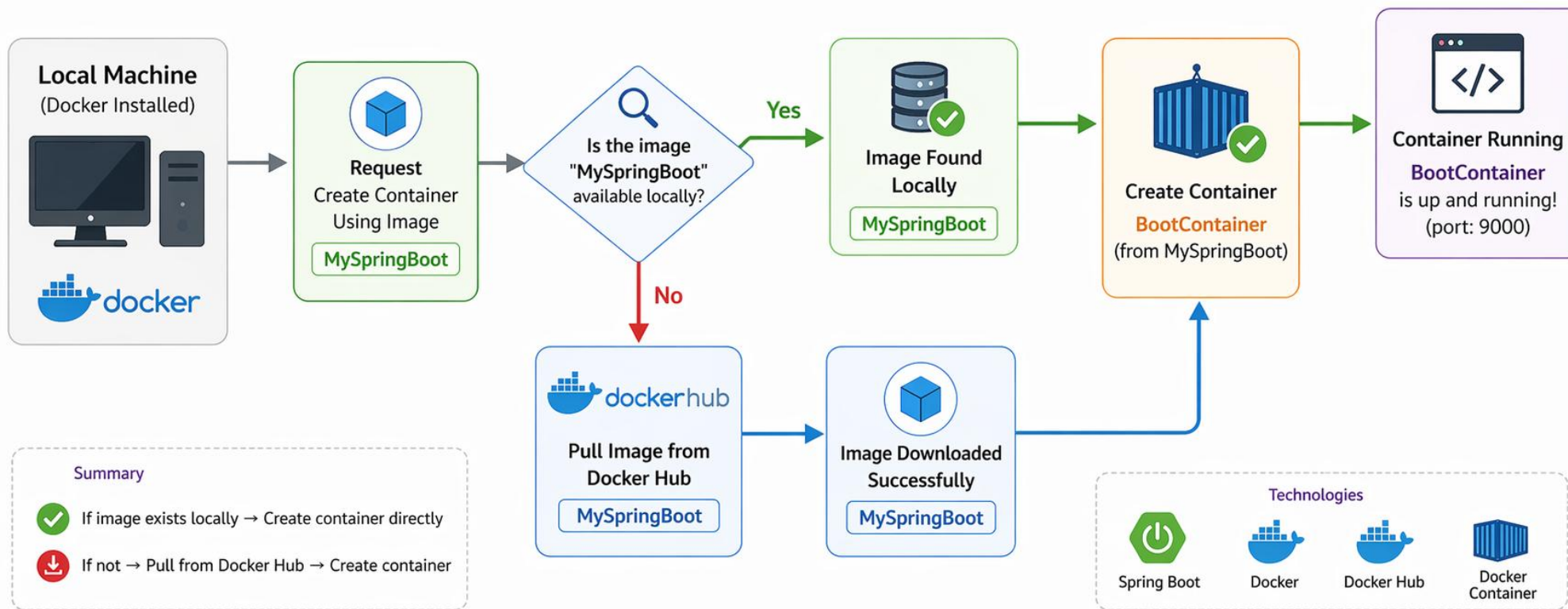


이미 생성되어 있는 Image를 사용하여 Container를 생성할 수 있습니다.



Create BootContainer from MySpringBoot Docker Image (Dynamic Flow)

If the image exists locally, create the container right away.
If not, pull it from Docker Hub, then create the container.



☰ 도커의 개요

☰ `컨테이너(Container)`

애플리케이션과 해당 실행에 필요한 모든 환경을 패키징한 실행 단위입니다.

컨테이너는 독립적으로 실행이 가능한 프로그램 목록을 저장해 놓은 공간입니다.

컨테이너를 생성하려면 Docker Image가 필요합니다.



컨테이너 예시



파이썬 3.x



Windows 11



mysql 8.x



워드 프레스



아파치



PHP

☰ 도커의 개요

☰ `도커(Docker)`

컨테이너 기술을 기반으로 애플리케이션과 그 실행 환경을 격리된 단위로 패키징하고 실행할 수 있는 플랫폼입니다.

이를 통해 개발, 테스트, 배포 과정에서 일관된 환경을 제공하여 "한 번 작성하면 어디서나 실행 가능(Write Once, Run Anywhere)한 환경을 구현합니다.

Docker는 개발자가 소프트웨어를 빌드하고, 패키징하고, 배포하는 방식을 혁신적으로 변화시켰으며, 가상 머신보다 가볍고 빠르며 유연한 솔루션으로 자리 잡았습니다.

항목	설명
Docker Daemon	Docker의 백그라운드 서비스로, 컨테이너와 이미지를 관리합니다. `dockerd`라는 이름으로 실행되며, Docker API를 통해 사용자 명령을 처리합니다.
Docker CLI	사용자가 Docker 데몬과 상호 작용할 수 있도록 명령을 제공합니다. 예: `docker run`, `docker build`, `docker ps` 등. docker에 명령어 전송을 위한 도구로는 Ubuntu terminal, cmd 창 등이 있습니다.
Docker Image	컨테이너 실행에 필요한 파일 시스템과 애플리케이션 환경을 포함하는 불변 템플릿. 레이어 기반으로 구성되어 있어 효율적인 저장과 네트워크 전송을 지원합니다. 예: Ubuntu 이미지, MySQL 이미지, WordPress 이미지 등.
Docker Container	이미지를 기반으로 실행되는 독립적인 단위로, 필요한 애플리케이션과 환경을 포함. 격리된 프로세스로 실행되며, 컨테이너 간에는 영향을 주지 않습니다.
Docker Registry	Docker 이미지를 저장하고 공유하는 중앙 저장소. Docker Hub가 가장 널리 사용되는 공개 레지스트리이며, 기업 환경에서는 Private Registry를 구성하기도 합니다.
Docker Engine	도커 컨테이너를 생성, 관리, 실행, 배포할 수 있는 소프트웨어 플랫폼입니다.
Docker Host	도커 엔진이 실행되는 물리적 서버 또는 가상 서버를 의미합니다.

☰ 도커의 개요

☰ `도커 호스트`

도커 엔진이 실행되는 물리적 서버 또는 가상 서버를 의미합니다.

도커 컨테이너, 이미지, 네트워크, 볼륨 등을 관리하고 실행하는 도커 데몬(`dockerd`)이 이 호스트 상에서 동작합니다.

이를 통해 컨테이너 기반 애플리케이션이 실행됩니다.

도커 호스트 구성 요소

항목	설명
Docker Engine	도커 데몬(`dockerd`)이 도커 호스트에서 동작하며, 컨테이너와 관련된 모든 작업을 처리.
운영 체제	도커 호스트는 리눅스, 윈도우, macOS와 같은 운영 체제에서 동작 가능. 리눅스 기반의 호스트는 네이티브 도커 엔진을 실행하며, 윈도우와 macOS는 Docker Desktop을 통해 도커를 실행.
컨테이너 런타임	도커 엔진에 의해 관리되는 컨테이너 실행 환경. 기본적으로 `containerd` 런타임을 포함.
네트워킹 구성	도커 호스트는 컨테이너 간 통신 및 외부 네트워크와의 연결을 처리. NAT, 브리지, 호스트 네트워크 등 다양한 네트워크 드라이버 지원.
스토리지	컨테이너 데이터와 도커 이미지를 저장하는 공간. 호스트의 파일 시스템 또는 외부 스토리지 볼륨을 활용. 컨테이너의 실행 여부와 상관 없이 항상 데이터 접근이 가능한 저장소입니다.

☰ 도커의 개요

☰ `도커 호스트

도커 엔진이 실행되는 물리적 서버 또는 가상 서버를 의미합니다.

도커 호스트의 역할

항목	설명
컨테이너 실행 환경 제공	도커 호스트는 도커 엔진을 실행하여 컨테이너가 동작할 수 있는 환경을 제공합니다.
자원 관리	CPU, 메모리, 네트워크, 스토리지 등의 하드웨어 자원을 컨테이너에 할당하고 관리.
컨테이너 네트워킹	각 컨테이너가 서로 통신하거나 외부 네트워크와 연결할 수 있도록 네트워크 인터페이스를 설정.
이미지 저장 및 배포	도커 이미지를 캐시로 보관하고 이를 기반으로 컨테이너를 생성.
보안 제공	컨테이너 간 자원 격리와 네트워크 격리를 통해 보안성을 높임.

☰ 도커의 개요

☰ `도커 이미지

컨테이너 실행을 위한 모든 필수 구성 요소를 포함하고 있는 읽기 전용 템플릿입니다.

필수 구성 요소는 애플리케이션 코드, 런타임, 라이브러리, 환경 설정 등을 의미합니다.

docker image는 container를 구성하고자 하는 데 필요한 파일 시스템과 실행할 애플리케이션 설정들을 하나로 합쳐 놓은 것입니다.

즉, 하나의 컨테이너를 생성하기 위한 템플릿 역할을 수행합니다.

docker container는 docker image를 기반으로 생성이 되며, 파일 시스템과 애플리케이션이 구체화되어 실행 되는 상태를 의미합니다.

특징	docker image	docker container
정의	애플리케이션 실행 환경과 종속성을 포함한 읽기 전용 템플릿	이미지 기반으로 실행된 상태의 독립적인 프로세스
상태	불변 (Immutable)	동적 (Dynamic)
생성 방법	Dockerfile, 레지스트리에서 다운로드	이미지를 기반으로 실행 (`docker run`)
목적	실행 가능한 환경 제공	애플리케이션 실행
크기	읽기 전용, 상대적으로 작음	컨테이너 레이어 포함으로 약간 더 클 수 있음

docker image

container를 생성하는 데 필요한
읽기 전용의 template

운영 체제

File System

Application File



create container

docker container

운영 체제

File System

Running Application

≡ 도커의 개요

컨테이너 기술을 기반으로 애플리케이션과 그 실행 환경을 격리된 단위로 패키징하고 실행할 수 있는 플랫폼입니다.

도커 개요

개념	설명	특징
이미지	프로그램을 위한 완성된 설치파일	실행되지 않음
컨테이너	이미지를 실행한 실제 프로그램	삭제하면 데이터 사라짐
볼륨	데이터 저장용 외부 하드	컨테이너와 독립, 데이터 유지

☰ 도커의 개요

☰ 네이밍 규칙

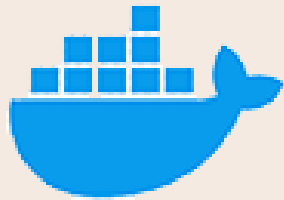
컨테이너, 이미지, 볼륨 이름에 접미사를 붙이면 프로젝트 관리가 훨씬 깔끔해집니다.

네이밍 규칙

리소스	추천 접미사	예시
이미지(Image)	-img	spring-img, mysql-img
컨테이너(Container)	-ctr	spring-ctr, mysql-ctr
볼륨(Volume)	-vol	mysql-vol, data-vol
네트워크(Network)	-net	backend-net, frontend-net
Deployment	-dep	spring-dep
Service	-svc	spring-svc
Pod	-pod	test-pod
ConfigMap	-cfg	spring-cfg
Secret	-sec	mysql-sec

Docker&Kubernetes

도커의 동작 원리



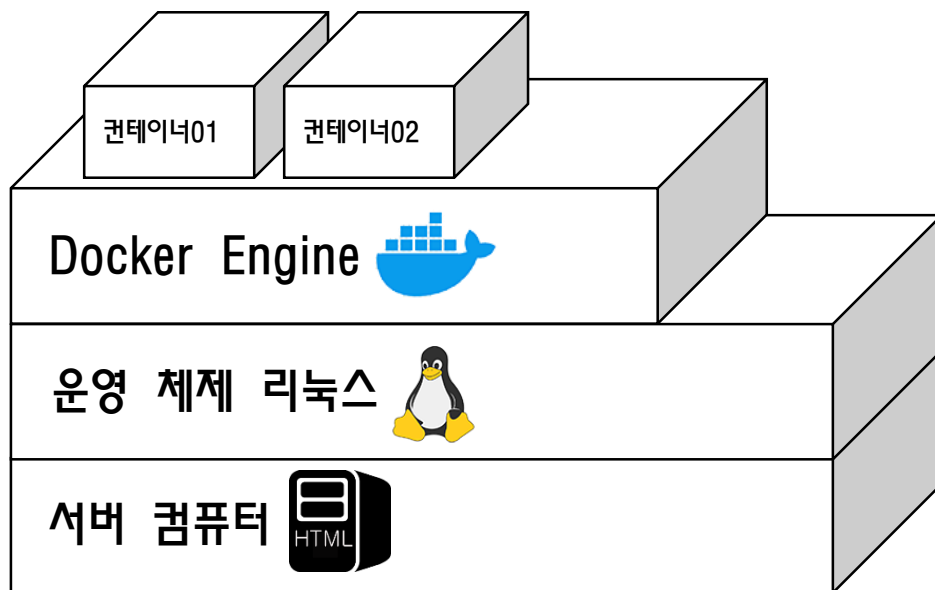
☰ 도커의 동작 원리

☰ 도커의 구조

docker와 container는 다음과 같은 구조로 구성되어 있습니다.

운영 체제 위에 Docker Engine이 동작합니다.

그 위에 Container가 동작하는 구조로 되어 있습니다.



도커의 구조

컨테이너는 도커 엔진 위에서 구동합니다.

프로그램 및 데이터는 컨테이너 내부에 존재합니다.

☰ 도커의 동작 원리

☰ 도커는 기본적으로 '리눅스용'

docker는 기본적으로 리눅스용 운영 체제에서 구동이 됩니다.

도커는 기본적으로 linux라는 운영 체제를 기반으로 설계가 되었습니다.

또한 컨테이너 내부에서 실행이 되는 소프트웨어 역시 리눅스용이어야 합니다.



0



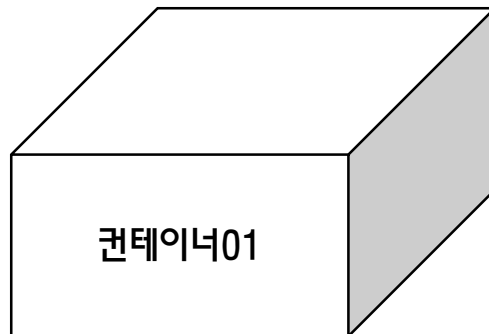
X



X



X



☰ 도커 허브와 이미지, 컨테이너

☰ 도커 이미지와 컨테이너

도커 이미지(Image)는 금형(Template)과 유사합니다.

겨울이 되면 붕어빵 틀에서 맛있는 붕어빵을 많이 만들어 냅니다.

이와 유사하게 `도커 이미지`는 도커 컨테이너를 만들어 내는 틀(Template)입니다.

하나의 도커 이미지를 이용하여 동일한 구성의 컨테이너를 여러 개를 만들어 낼 수 있습니다.



하나의 이미지로 동일한 구성 정보를 담고 있는 여러 개의 컨테이너를 만들 수 있습니다.

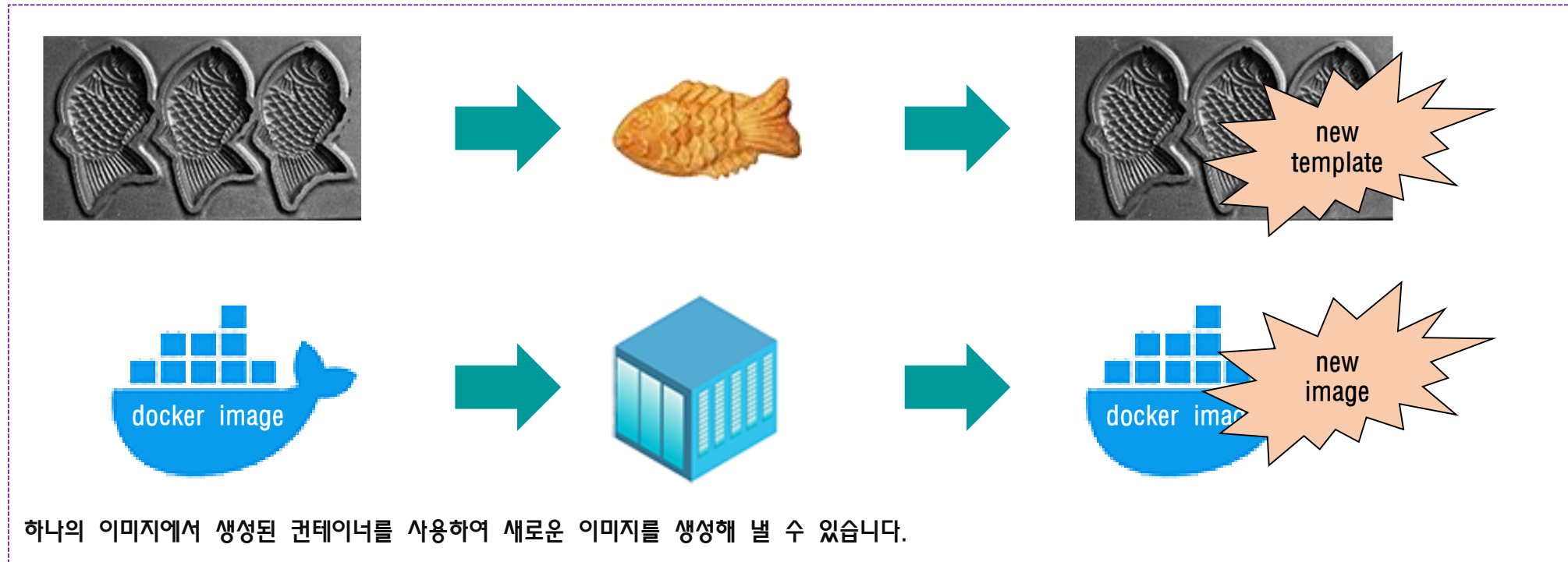
☰ 도커 허브와 이미지, 컨테이너

☰ 도커 이미지와 컨테이너

컨테이너로도 이미지 생성을 할 수 있습니다.

도커 이미지를 생성된 컨테이너를 사용하여 조금 변형된 이미지를 새로이 생성 가능합니다.

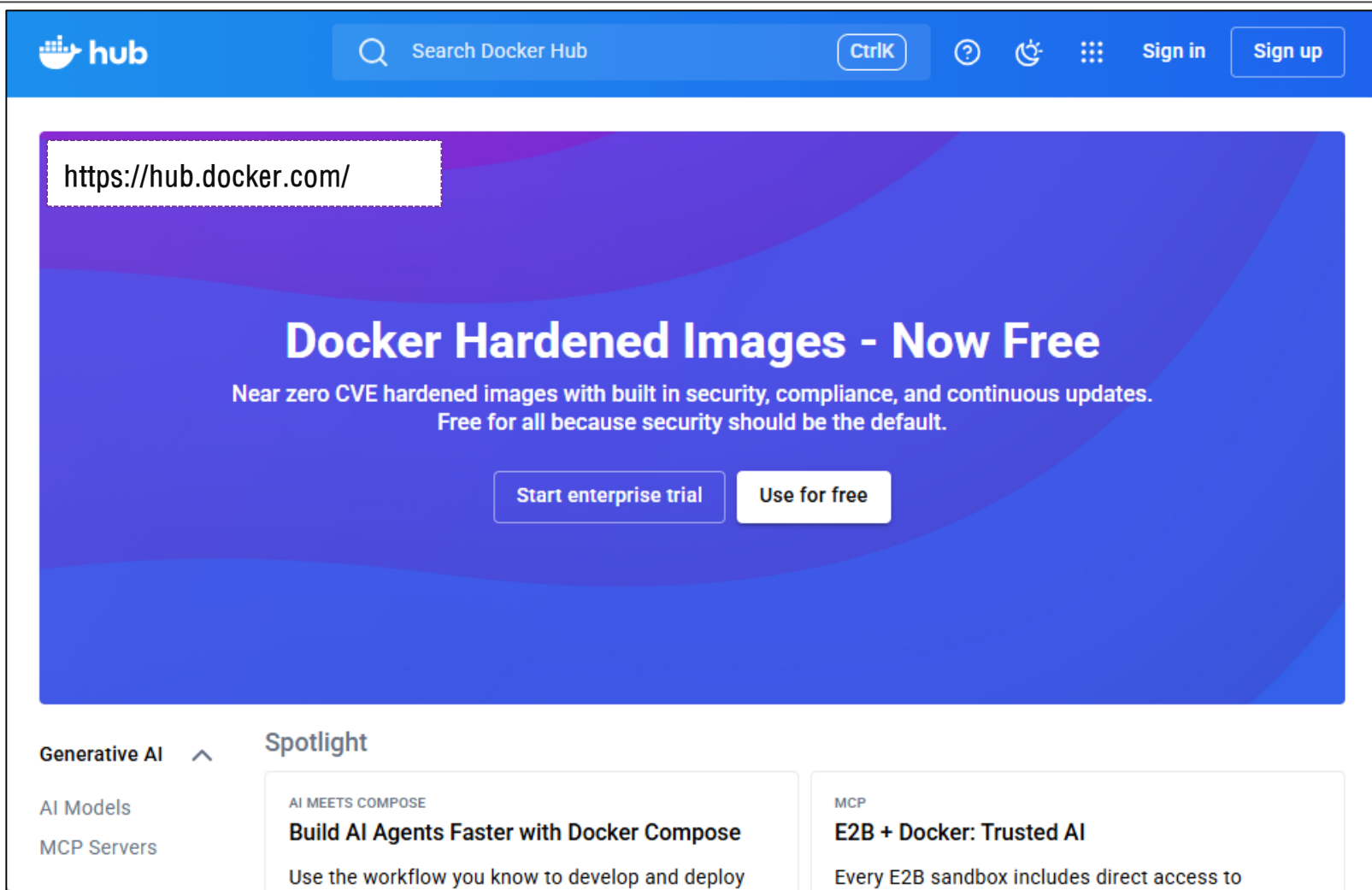
변형이란, 기존 이미지에 새로운 소프트웨어를 추가하든지 설정 값을 변경하는 정도의 작업을 의미합니다.



☰ 도커 허브와 이미지, 컨테이너

☰ `도커 허브

도커 컨테이너 이미지를 저장하고 공유하기 위한 클라우드 기반의 레지스트리 서비스입니다.



☰ 도커 허브와 이미지, 컨테이너

☰ `도커 허브

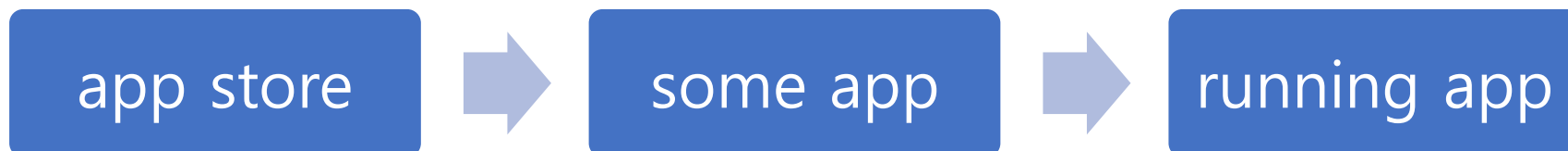
도커 컨테이너 이미지를 저장하고 공유하기 위한 클라우드 기반의 레지스트리 서비스입니다.

도커 Hub는 휴대폰의 구글 플레이 스토어와 유사한 개념으로, **공개된 컨테이너 이미지들이 모여 있는 저장소**입니다.

공개된 컨테이너 이미지가 모여 있는 저장소로써 이미지를 upload/download가 가능합니다.

도커 엔진과 통합되어 사용자가 이미지 생성, 관리, 배포를 손쉽게 할 수 있도록 지원하는 공식 도커 Registry입니다.

App 구현 절차



container 구동 절차



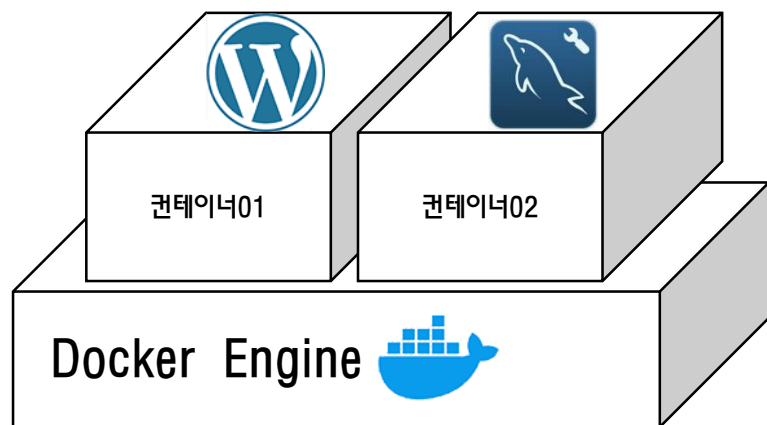
☰ 도커 허브와 이미지, 컨테이너

☰ 도커 허브

도커 허브에서 우리가 원하는 이미지를 다운로드 할 수 있습니다.

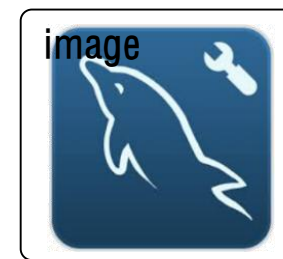
도커 허브에는 다양한 유형의 이미지가 제공이 됩니다.

누구든지 이미지를 등록하고 공개 가능합니다.



← 이미지 내려 받기

Docker Hub



☰ 도커 허브와 이미지, 컨테이너

☰ 도커 허브

Docker Hub에서 저장소를 만드는 절차입니다.

The screenshot shows the Docker Hub interface for a user named 'ugcadman'. The left sidebar contains a menu with options: Repositories (highlighted), Hardened Images, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main content area has a blue header with 'Welcome to Docker' and a 'Repository 생성' (Create Repository) button. Below the header, there is a text box with the URL 'https://hub.docker.com/repositories/your_register' and the text '사이트에서 Repository를 생성합니다.' (Create Repository on the site). A 'Download for Windows' button is also visible. At the bottom, there are three cards: 'Create a Repository' (highlighted with a red box), 'Docker Hub Basics', and 'Language-Specific Guides'. The 'Create a Repository' card contains the text 'Push container images to a repository on Docker Hub.' The bottom of the page features the text 'Access the world's largest library of container images'.

Repository 생성

Welcome to Docker

https://hub.docker.com/repositories/your_register

사이트에서 Repository를 생성합니다.

Download for Windows

Also available for [Mac](#) and [Linux](#)

Create a Repository
Push container images to a repository on Docker Hub.

Docker Hub Basics
Watch the guide on how to create and push your first image into a Docker Hub repository.

Language-Specific Guides
Learn how to containerize language-specific applications using Docker.

Access the world's largest library of container images

☰ 도커 허브와 이미지, 컨테이너

☰ 도커 허브

Docker Hub에서 저장소를 만드는 예시입니다.

[Repositories](#) / [Create](#)

Using 0 of 1 private repositories. [Get more](#)

생성 예시

Create repository

Repository Name *

cicd-springboot



Short description

Spring Boot CI/CD Test

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. [Get more](#)

Public



Appears in Docker Hub search results

Private



Only visible to you

Cancel

Create

Pushing images

항목	설명
Repository Name	cicd-springboot
Visibility	Public (학습용이면 Public)
Description	Spring Boot CI/CD Test

Make sure to replace `tagname` with your desired image repository tag.

☰ 도커 허브와 이미지, 컨테이너

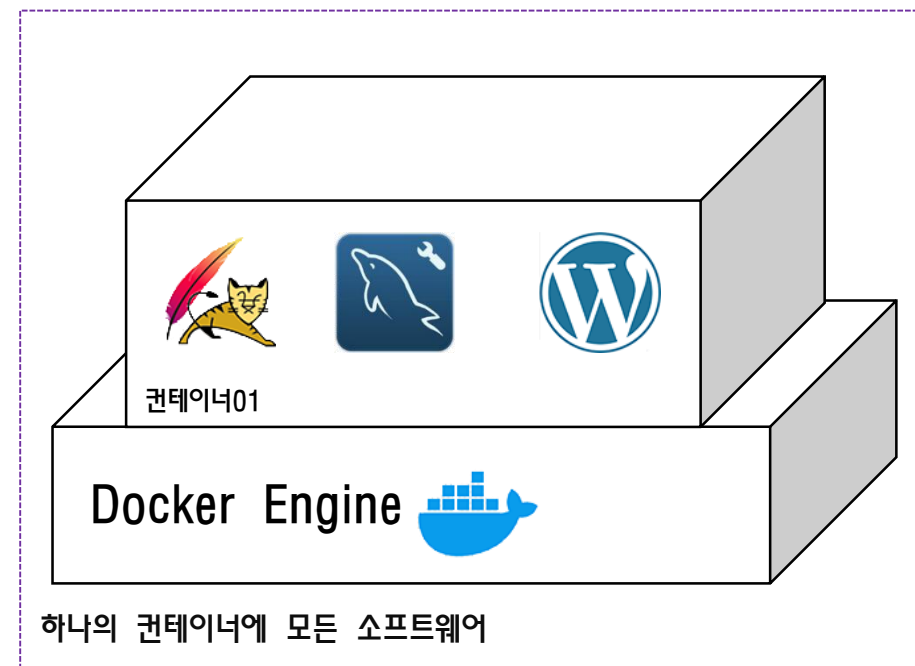
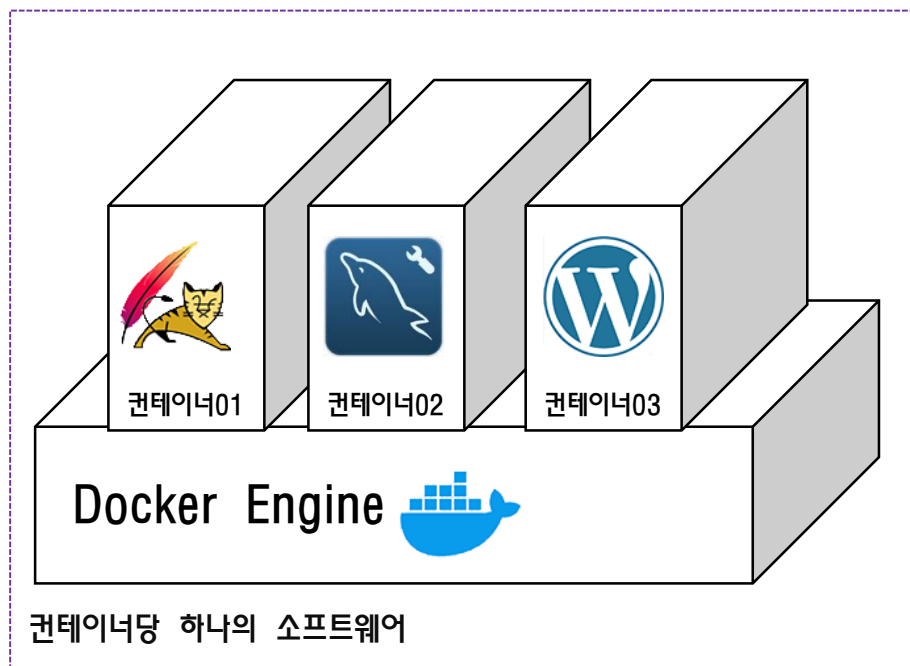
☰ 도커의 구조

컨테이너는 다양한 형태로 조합이 가능합니다.

예를 들어서 워드 프레스를 구축하려면, 아파치와 MySQL 그리고 WordPress가 필요합니다.

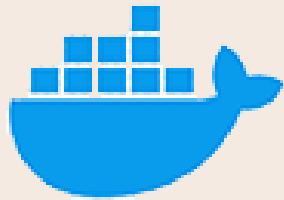
이를 구축하는 방법은 각각의 소프트웨어를 별도의 컨테이너로 구성하는 방법이 있습니다.

다른 방법은 모든 관련 소프트웨어를 하나의 컨테이너에 포함시켜 구현하는 방법도 있습니다.



Docker&Kubernetes

컨테이너 실행해 보기



☰ 컨테이너 실행해 보기

☰ 도커 엔진을 시작/종료하는 방법

도커 엔진을 설치와 함께 실행이 되며, 계속 동작 상태로 남아 있습니다.

docker desktop은 도커 엔진을 자동으로 실행하도록 설정이 되어 있습니다.

Docker Engine



자동 실행 설정이 활성화 되어 있습니다.
즉, 부팅이 될때마다 자동으로 실행이 됩니다.

Container



컨테이너를 실행 시킬 별도의 스크립트가 필요
합니다.

≡ 컨테이너 실행해 보기

≡ 컨테이너의 기본적인 사용 방법

컨테이너를 사용해 보기 위한 docker 명령에 대하여 살펴 보겠습니다.

컨테이너를 사용하기 위한 기본 명령어에 대하여 살펴 보겠습니다.

명령 프롬프트나 Terminl 창을 이용하여 명령어를 입력하면 됩니다.

docker 명령어

최신 버전에서 커맨드가 재편되면서 상위와 하위 커맨드 개념이 생겼습니다.

이전 버전과의 호환성을 위해서 이전 방식의 문법도 호환이 가능합니다.

docker 커맨드 (옵션) 대상 (매개 변수)

```
docker container run --name apache81 -d -p 8881:80 httpd
```

항목	설명
커맨드	상위 커맨드(`무엇을`)와 하위 커맨드(`어떻게`) 등에 해당하는 영역입니다.
옵션	커맨드에 좀더 세부적인 설정을 지정하는 영역입니다. 예를 들어서 백그라운드 실행은 <code>-d</code> 옵션, 포트 번호 지정을 하려면 <code>-p</code> 등입니다.
대상	작업을 수행할 대상으로 Container 또는 Image 등의 이름을 지정합니다.
매개 변수	대상에 전달할 값을 지정합니다. 문자 코드 또는 포트 번호 등을 전달할 수 있습니다.

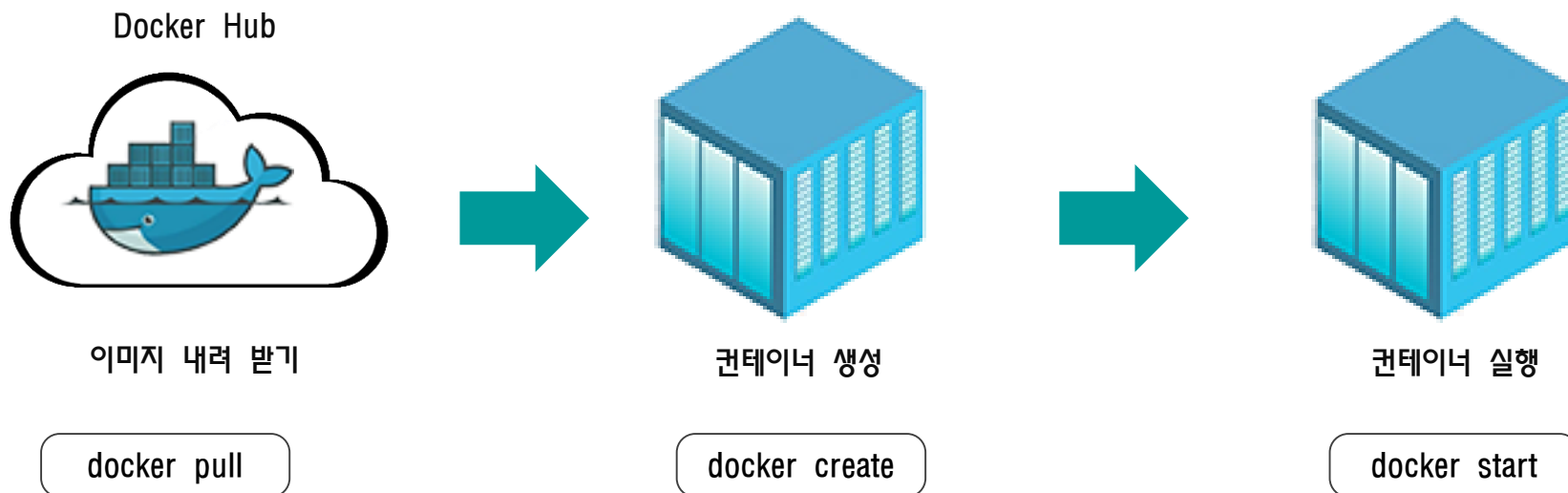
≡ 컨테이너 실행해 보기

≡ 컨테이너의 생성과 삭제/실행/정지

컨테이너를 생성/삭제/실행/정지 등의 명령어를 학습해 봅니다.

도커 컨테이너를 실행하고자 하는 경우 [docker container run] 명령어를 사용합니다.

이 명령어는 다음과 같이 3개의 명령어를 합친 결과와 동일한 결과를 보여 줍니다.



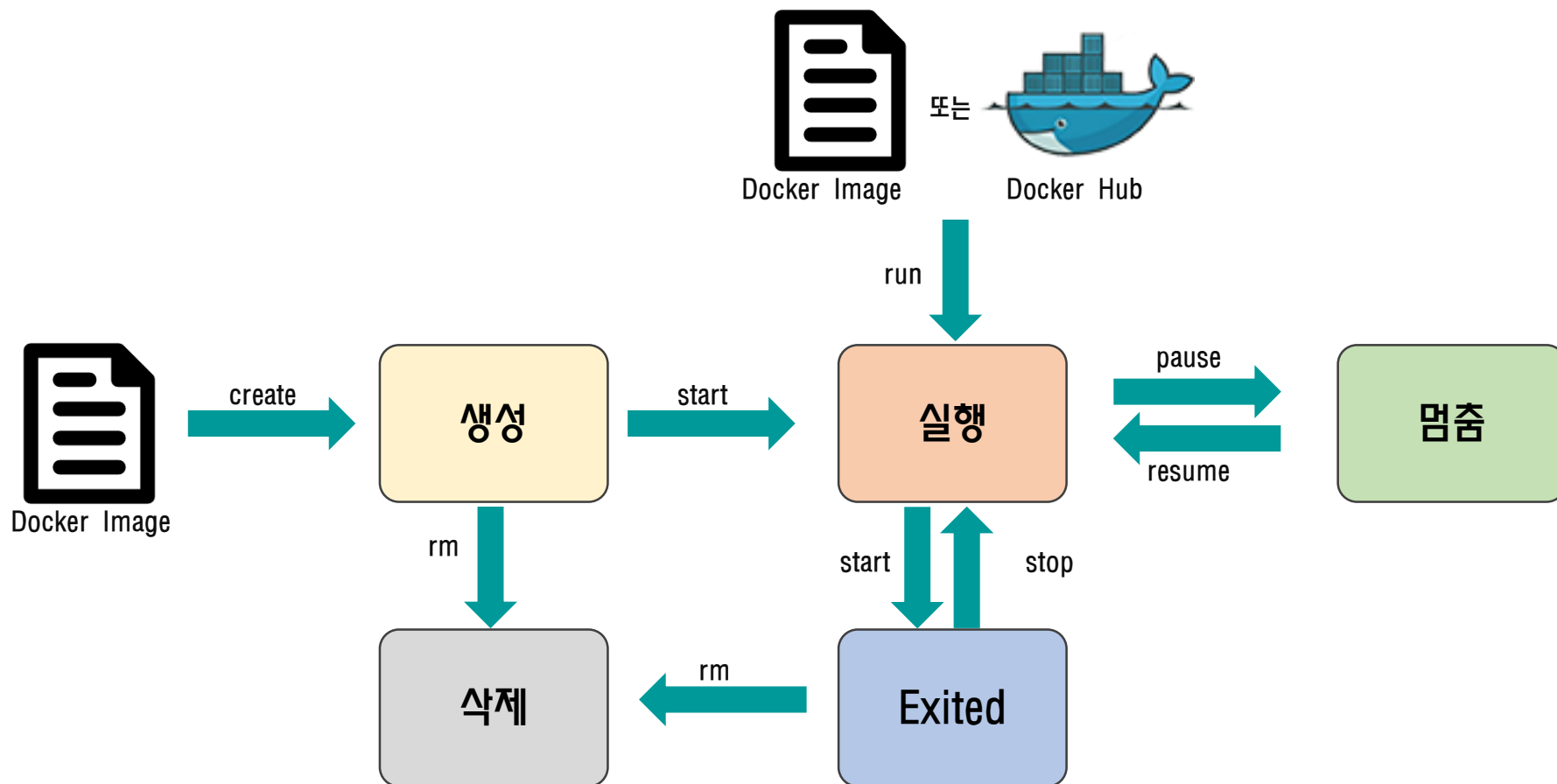
≡ 컨테이너 실행해 보기

≡ 컨테이너의 생성과 삭제/실행/정지

컨테이너는 생명 주기를 가지고 있습니다.

도커 컨테이너는 필요에 의하여 생성이 되고, 사용이 되고 마지막에는 삭제가 됩니다.

이러한 과정을 컨테이너의 생명 주기라고 합니다.

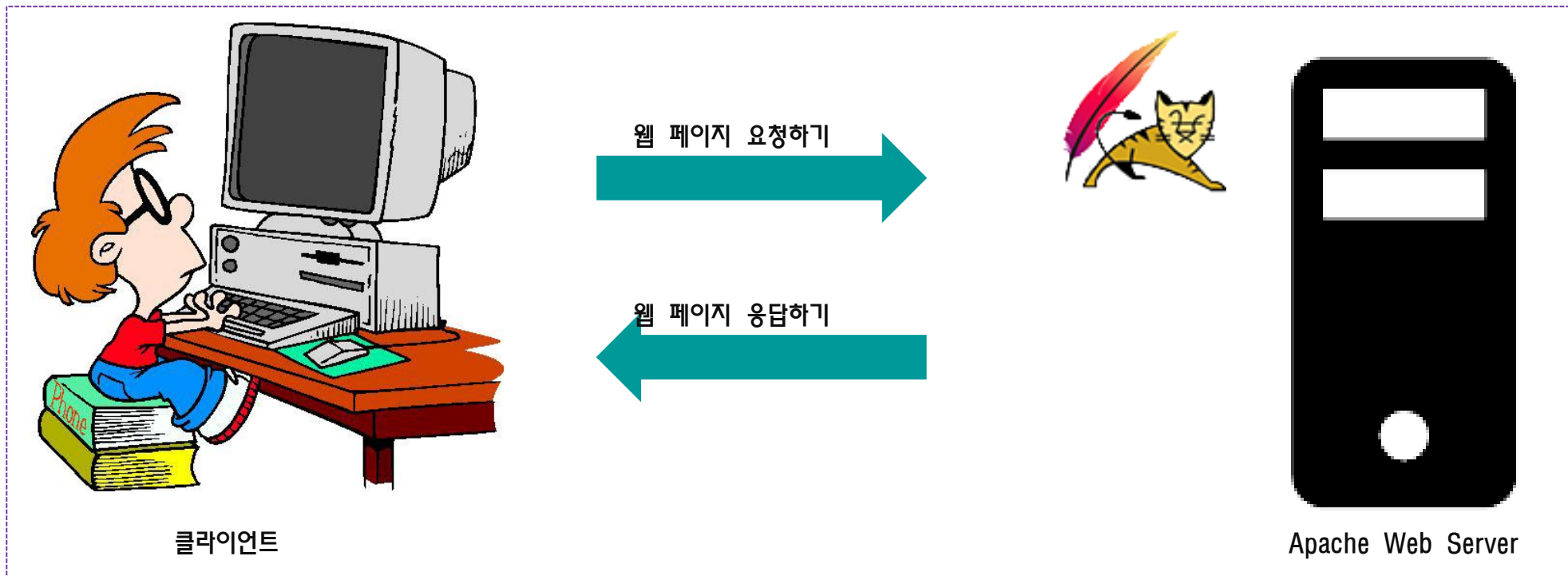


≡ 컨테이너 실행해 보기

≡ 컨테이너와의 통신

아파치는 웹 서버 기능을 제공해 주는 소프트웨어입니다.

Apache는 일반적으로 Apache Software Foundation(ASF)에서 개발하고 관리하는 다양한 오픈 소스 소프트웨어 프로젝트를 지칭합니다.
이 중 가장 잘 알려진 것은 Apache HTTP Server(Apache 웹 서버)로, 웹 애플리케이션과 웹사이트를 서비스하기 위해 사용되는 웹 서버 소프트웨어입니다.



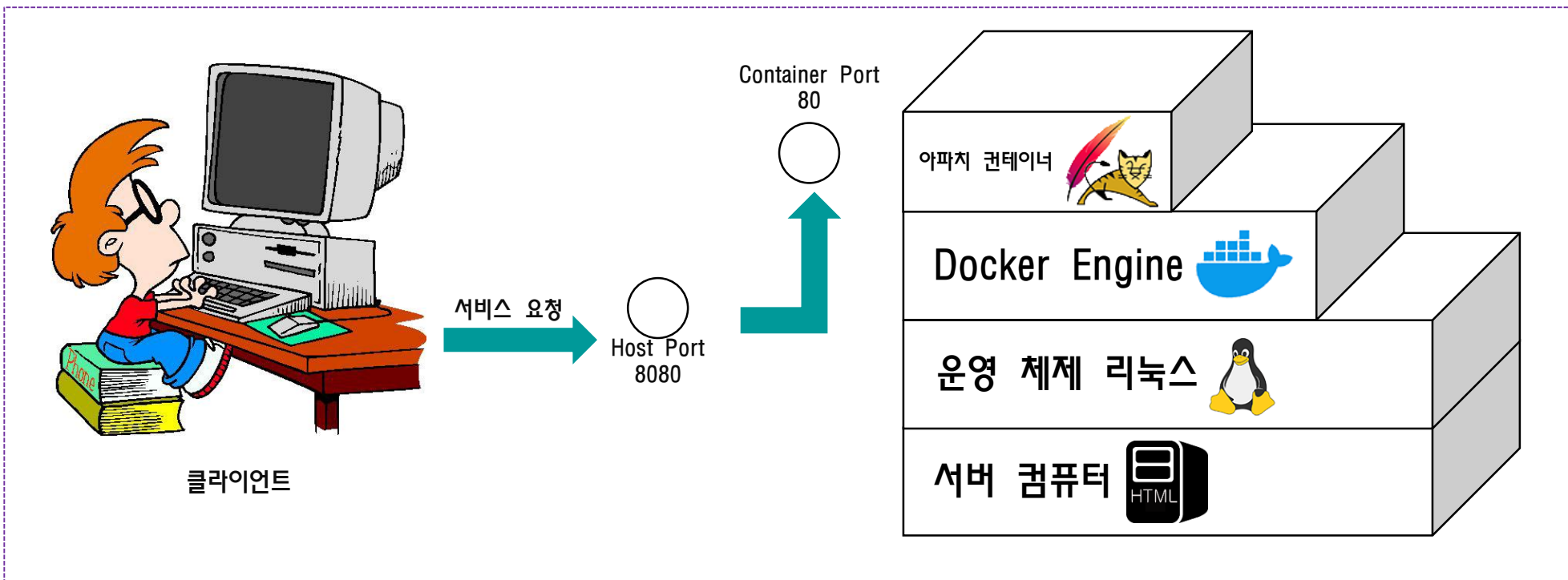
≡ 컨테이너 실행해 보기

≡ 컨테이너와의 통신

아파치는 웹 서버 기능을 제공해 주는 소프트웨어입니다.

클라이언트가 외부에서 아파치 웹 서버 요청을 하면 실행 중인 컴퓨터(host라고 함)의 포트 번호 8080가 이것을 접수합니다.

이상이 없음을 확인한 다음, 해당 컨테이너의 포트 번트 80과 연결합니다.



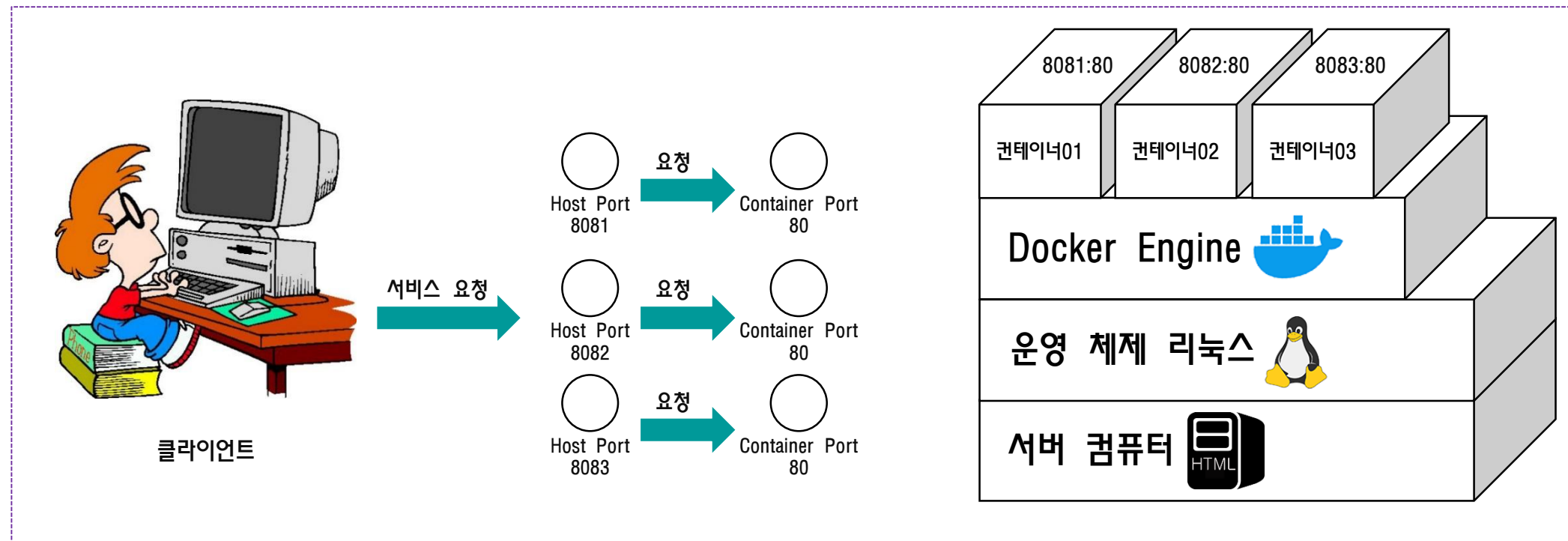
≡ 컨테이너 실행해 보기

≡ 컨테이너와의 통신

하나의 컨테이너에 여러 개의 웹 서버가 구동 가능합니다.

이와 같은 경우 host의 포트 번호는 서로 상이해야 합니다.

예를 들어서, 3개의 웹 서버가 구동이 된다면 다음과 같습니다.



≡ 컨테이너 실행해 보기

≡ 컨테이너 생성에 익숙해지기

다양한 유형의 컨테이너에 대하여 살펴 보도록 하겠습니다.

컨테이너에는 다양한 종류의 항목들이 존재합니다.
소프트웨어의 개수 그 이상만큼 다양하다고 보시면 됩니다.

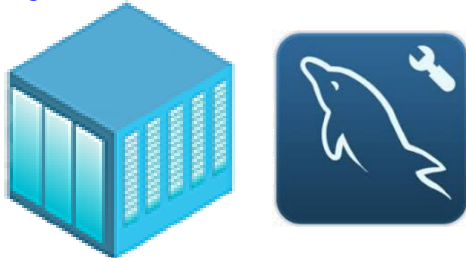
WordPress



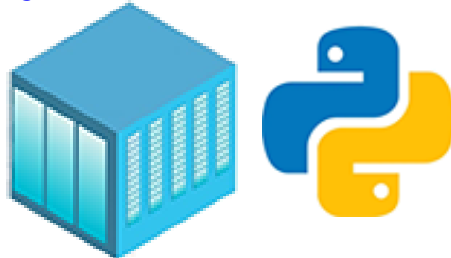
Apache



MySQL



Python



≡ 컨테이너 실행해 보기

≡ `도커 이미지

다음은 널리 사용되고 있는 대표적인 Docker 이미지들과 그 주요 용도입니다.

이름	용도	특징
ubuntu	기본적인 리눅스 환경 제공. 다양한 애플리케이션의 개발 및 테스트 환경 구축.	경량화된 이미지로 빠른 실행 가능. 다양한 태그(버전별) 지원: `ubuntu:20.04`, `ubuntu:latest` 등.
nginx	웹 서버 설정 및 배포. 로드 밸런싱 및 리버스 프록시 서버 구축.	빠르고 안정적인 웹 서버 이미지. 설정이 간단하며, 다양한 태그(버전별) 제공.
mysql	데이터베이스 서버 환경 구축에 필요한 컨테이너입니다. 개발 및 테스트용 데이터베이스를 제공합니다.	데이터 볼륨을 사용하여 데이터 지속성 유지. 다양한 태그로 버전 선택 가능: `mysql:8.0`, `mysql:5.7`.
python	Python 애플리케이션 개발 및 실행 환경. 데이터 분석, 웹 애플리케이션 개발 등에 사용.	Python 인터프리터와 패키지 관리 툴(pip)을 포함. 다양한 버전 제공: `python:3.10`, `python:3.11`.
debian	다양한 애플리케이션을 실행하기 위한 리눅스 환경. 기본 운영 체제 이미지로 사용.	안정적이고 유연한 환경 제공. 다양한 버전과 설정 가능.
elasticsearch	분산 검색 및 분석 엔진 구축. 로그 데이터 및 실시간 검색 시스템 구현.	Kibana와 함께 사용 가능. 대규모 데이터 처리 지원.
openjdk	Java 애플리케이션 실행 환경. 백엔드 애플리케이션 및 스프링 부트 프로젝트.	다양한 JDK 버전 제공: `openjdk:8`, `openjdk:17`.
tiangolo/uwsgi-nginx-flask	Flask 기반의 웹 애플리케이션 배포.	uwsgi와 Nginx를 포함하여 고성능 환경 제공.
httpd	Apache 웹 서버를 실행하기 위한 Docker 이미지. 웹 서버를 컨테이너화하여 간단히 애플리케이션 배포와 테스트. 오픈 소스 HTTP 서버로, 웹 페이지를 호스팅하는 데 널리 사용	경량 웹 서버 환경 제공. 다양한 태그 지원.

≡ 컨테이너 실행해 보기

≡ `도커 이미지

다음은 리눅스 운영 체제만 담겨서 제동되는 컨테이너 목록입니다.

리눅스 운영 체제 컨테이너에는 컨테이너 속 파일 시스템을 다루는 것을 전제로 하므로 매개 변수로 셸 명령어를 지정합니다.

이미지 이름	컨테이너의 내용	주요 옵션 및 매개 변수
ubuntu	우분투	-d 없이 -it 옵션만 사용. /bin/bash 등 셸 명령어를 지정합니다.
centos	CentOS	-d 없이 -it 옵션만 사용. /bin/bash 등 셸 명령어를 지정합니다.
debian	데비안	-d 없이 -it 옵션만 사용. /bin/bash 등 셸 명령어를 지정합니다.
fedora	페도라	-d 없이 -it 옵션만 사용. /bin/bash 등 셸 명령어를 지정합니다.
busybox	BusyBox	-d 없이 -it 옵션만 사용. /bin/bash 등 셸 명령어를 지정합니다.
alpine	알파인 리눅스	-d 없이 -it 옵션만 사용. /bin/bash 등 셸 명령어를 지정합니다.

≡ 컨테이너 실행해 보기

≡ `도커 이미지

다음은 웹서버/데이터 베이스 서버용 컨테이너 목록입니다.

가장 많이 사용하는 웹서버는 Apache가 있고, 이 외에도 많이 사용되는 항목은 엔진x(nginx)가 있습니다.

웹 서버는 통신을 전제로 하므로, Port 번호를 반드시 지정해 주어야 합니다.

데이터 베이스 서버는 MySql, PostgreSQL, MariaDB 등이 있습니다.

데이터 베이스 관리 소프트웨어는 관리자용 root password를 반드시 지정해 주어야 합니다.

이미지 이름	컨테이너의 내용	주요 옵션 및 매개 변수
httpd	아파치	-d로 백그라운드로 실행, -p로 포트 번호 지정
nginx	엔진x	-d로 백그라운드로 실행, -p로 포트 번호 지정
mysql	MySql	-d를 사용, 실행시 -e MYSQL_ROOT_PASSWORD와 같이 root password를 지정
postgres	PostgreSQL	-d를 사용, 실행시 -e POSTGRES_ROOT_PASSWORD와 같이 root password를 지정
mariadb	MariaDB	-d를 사용, 실행시 -e MYSQL_ROOT_PASSWORD와 같이 root password를 지정

≡ 컨테이너 실행해 보기

≡ `도커 이미지

다음은 프로그램 실행을 위한 런타임과 그외 소프트웨어용 컨테이너 목록입니다.

특정 프로그램을 실행하려면 해당 프로그램의 런타임이 필요합니다.

다음은 런타임을 위한 소프트웨어용 컨테이너 목록입니다.

이미지 이름	컨테이너의 내용	주요 옵션 및 매개 변수
oeponjdk	자바 런타임	-d를 사용하지 않고, 매개 변수로 java 명령 등을 지정해 도구 형태로 사용합니다.
python	파이썬 런타임	-d를 사용하지 않고, 매개 변수로 python 명령 등을 지정해 도구 형태로 사용합니다.
php	PHP 런타임	웹 서버가 포함된 것과 실행 명령만 포함된 것으로 나뉘어 제공됩니다.
ruby	루비 런타임	웹 서버가 포함된 것과 실행 명령만 포함된 것으로 나뉘어 제공됩니다.
perl	펄 런타임	-d를 사용하지 않고, 매개 변수로 perl 명령 등을 지정해 도구 형태로 사용합니다.
gcc	C/C++ 컴파일러	-d를 사용하지 않고, 매개 변수로 gcc 명령 등을 지정해 도구 형태로 사용합니다.
node	Node.js	-d를 사용하지 않고, 매개 변수로 app 명령 등을 지정해 도구 형태로 사용합니다.
registry	도커 레지스트리	-d를 사용하여 백그라운드로 실행하며, -p를 사용하여 포트 번호를 지정합니다.
wordpress	워드 프레스	-d를 사용하여 백그라운드로 실행하며, -p를 사용하여 포트 번호를 지정합니다. MySQL 또는 MariaDB가 필요하며, -e 옵션으로 접속에 필요한 패스워드를 설정합니다.
nextcloud	NextCloud	-d를 사용하여 백그라운드로 실행하며, -p를 사용하여 포트 번호를 지정합니다.
redmine	Redmine	-d를 사용하여 백그라운드로 실행하며, -p를 사용하여 포트 번호를 지정합니다. MySQL 또는 PostgreSQL이 필요합니다.

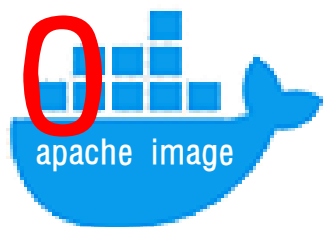
≡ 컨테이너 실행해 보기

≡ 이미지 삭제

실습에 사용했던 이미지 정보들을 삭제해 보겠습니다.

이전 실습에서 사용한 컨테이너를 삭제하더라도, 이미지는 삭제되지 않습니다.

많은 이미지가 하드 디스크에 존재하면 스토리지 용량에 대한 압박을 받을 수 있으므로, 사용되지 않는 이미지는 삭제할 필요가 있습니다.



이전에 사용했던 Apache 컨테이너 3개가 모두 삭제된 상황이므로 Apache Image는 삭제가 가능합니다.



이전에 사용했던 Nginx 컨테이너 2개 중에서 1개만 삭제된 상황입니다.

해당 Nginx 이미지를 사용하여 현재 1개가 실행되고 있으므로 Nginx 이미지는 삭제가 불가능합니다.

≡ 컨테이너 실행해 보기

≡ 이미지 목록 확인/삭제

실습에 사용했던 이미지 정보들을 확인 후 필요 없는 항목은 삭제해 보도록 하겠습니다.

이미지를 삭제하려면 이미지의 이름이나 이미지의 id를 알고 있어야 합니다.

이미지 목록을 확인하는 방법과 해당 이미지를 삭제하는 방법을 살펴 봅니다.

image 관련 명령어

rem docker image 정보를 확인합니다.

rem 이미지의 이름 이외에도 태그(버전 정보), 이미지 ID 등의 정보를 확인할 수 있습니다.

docker image ls

항목	설명
REPOSITORY	이미지의 이름
TAG	버전 정보, 다운로드시 별도의 지정이 없으면 최신 버전(latest)을 내려 받습니다.
IMAGE ID	이미지 식별자, 총 64글자이지만 실제 보이는 항목은 12글자(unique)만 보여 줍니다.
CREATED	이미지가 생성된 후 경과 시간을 의미합니다.
SIZE	이미지의 전체 용량 정보입니다.

≡ 컨테이너 실행해 보기

≡ 이미지 목록 확인/삭제

실습에 사용했던 이미지 정보들을 확인 후 필요 없는 항목은 삭제해 보도록 하겠습니다.

이미지를 삭제하려면 이미지의 이름이나 이미지의 id를 알고 있어야 합니다.

image 관련 명령어

rm 명령어를 사용하여 이미지 삭제가 가능합니다.

이미지 이름 또는 이미지 ID를 이용하여 삭제하도록 합니다.

다음 예시는 3개의 이미지를 inline 스타일로 작성하여 동시에 삭제하는 명령어입니다.

```
docker image rm nginx mysql httpd
```

≡ 컨테이너 실행해 보기

≡ Docker 재시작 정책(`restart policy`)

부팅 후에 컨테이너가 자동으로 다시 켜지게 하는 간단한 방법입니다.

재시작 정책

```
# WordPress 컨테이너
docker update --restart=always wordpress01

# MySQL 컨테이너
docker update --restart=always mysql01
```

설정 확인 방법

```
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' wordpress01

docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' mysql01
```

RedMine

프로젝트 관리 + 이슈(버그/할 일) 추적을 위한 웹 기반 협업 도구입니다.

무엇을 할 수 있나요?

이슈(작업) 관리

버그, 기능 요청, 작업(Task) 등록

상태 관리: 신규 → 진행 중 → 완료

담당자 지정, 우선순위 설정

프로젝트 관리

프로젝트별 이슈 분리 관리

일정 관리 (마일스톤, 버전)

진행률 확인

일정 & 진척도

간트 차트

캘린더

작업 소요 시간(Time Tracking)

협업 기능

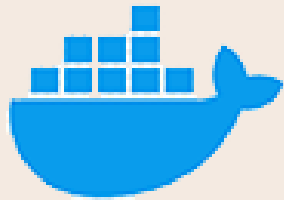
사용자 권한 관리

댓글 / 이력 관리

파일 첨부

Docker&Kubernetes

컨테이너들간의 연동



≡ 컨테이너들간의 연동

≡ 워드프레스 사이트 구성 및 구축

워드프레스에 대한 사이트 구성 및 구축을 실습해 보도록 합니다.

일반적으로 워드 프레스는 아파치, 데이터 베이스, PHP 런타임 등을 동시에 사용하여 구축을 많이 합니다.

워드 프레스 구축



워드프레스를 사용하기 위하여 2개의 컨테이너를 사용하도록 합니다.

≡ 컨테이너들간의 연동

≡ 도커 네트워크 생성 및 삭제

가상 네트워크란 2개 이상의 컨테이너를 연결하기 위한 네트워크를 의미합니다.

가상 네트워크를 생성하는 명령어는 `docker network` 입니다.

다음은 이 명령에 추가되는 하위 커맨드 목록입니다.

커맨드	내용
<code>connect</code>	네트워크에 컨테이너를 접속시킵니다.
<code>disconnect</code>	네트워크에 컨테이너를 접속 종료합니다.
<code>create</code>	신규 네트워크를 생성합니다.
<code>inspect</code>	네트워크의 상세 정보를 JSON 형식으로 확인할 수 있습니다.
<code>ls</code>	네트워크 목록을 확인합니다.
<code>prune</code>	접속된 컨테이너 개수가 0인 네트워크들을 삭제합니다.
<code>rm</code>	지정한 네트워크를 삭제합니다.

≡ 컨테이너들간의 연동

≡ `MySQL 컨테이너 관련 옵션/매개 변수

MySQL 컨테이너 생성과 관련된 옵션 및 매개 변수들은 다음과 같습니다.

관련 옵션

사용 예시	설명
--name asdf	asdf라는 이름으로 컨테이너를 생성합니다.
-dit	백그라운드에서 실행 및 키보드를 사용하여 컨테이너 내부의 파일 시스템을 조작합니다.
--net=network01	네트워크 이름을 network01으로 지정합니다.
-e MYSQL_ROOT_PASSWORD=root	루트 사용자의 패스워드를 지정합니다.
-e MYSQL_DATABASE=coffee	사용하고자 하는 데이터 베이스 이름을 지정합니다.
-e MYSQL_USER=myuser	사용자 이름을 myuser으로 지정합니다.
-e MYSQL_PASSWORD=mypassword	사용자의 비밀 번호를 mypassword으로 지정합니다.

매개 변수

항목	매개 변수	의미
문자열 인코딩	--character-set-server=utf8mb4	문자열 인코딩으로 UTF-8을 사용합니다.
정렬 순서	--collation-server=utf8mb4_unicode_ci	정렬 순서로 UTF-8을 따릅니다.
인증 방식	이 항목은 굳이 사용하지 않아도 됩니다.	

≡ 컨테이너들간의 연동

≡ WordPress 컨테이너 관련 옵션/매개 변수

워드프레스 컨테이너 생성과 관련된 옵션 및 매개 변수들은 다음과 같습니다.

관련 옵션

사용 예시	설명
--name asdf	asdf라는 이름으로 컨테이너를 생성합니다.
-dit	백그라운드에서 실행 및 키보드를 사용하여 컨테이너 내부의 파일 시스템을 조작합니다.
--net=network01	네트워크 이름을 network01으로 지정합니다.
-p=8085:80	포트 번호를 지정합니다.
-e WORDPRESS_DB_HOST=mysql01	데이터 베이스 컨테이너 이름을 지정합니다.
-e WORDPRESS_DB_NAME=coffee	사용하고자 하는 데이터 베이스 이름을 coffee라는 이름으로 지정합니다.
-e WORDPRESS_DB_USER=myuser	사용자 이름을 myuser으로 지정합니다.
-e WORDPRESS_DB_PASSWORD=mypassword	사용자의 비밀번호를 mypassword으로 지정합니다.

≡ 컨테이너들간의 연동

≡ Spring 컨테이너 관련 옵션/매개 변수

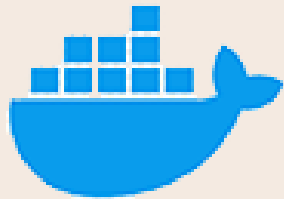
SPRING_으로 시작하는 모든 항목들은 스프링 부트(Spring Boot)에서 사용하는 환경 변수입니다.

관련 옵션

사용 예시	설명
SPRING_PROFILES_ACTIVE=linux	스프링 부트의 활성 프로파일 설정 스프링 부트는 application.properties 외에 application-프로파일명.properties 파일을 만들 수 있습니다.
SPRING_DATASOURCE_URL=jdbc:mysql://mysql_container:3306/dockerdb	스프링 부트의 DB 접속 URL 설정 이 값은 application.properties 안의 아래 설정과 동일한 의미입니다. spring.datasource.url=jdbc:mysql://mysql_container:3306/dockerdb
SPRING_DATASOURCE_USERNAME=spring	DB 접속 계정(username) application.properties에서는 spring.datasource.username=spring
SPRING_DATASOURCE_PASSWORD=Spring@1234	DB 접속 비밀번호(password) properties 파일의 spring.datasource.password=Spring@1234

Docker&Kubernetes

실전 컨테이너 사용법

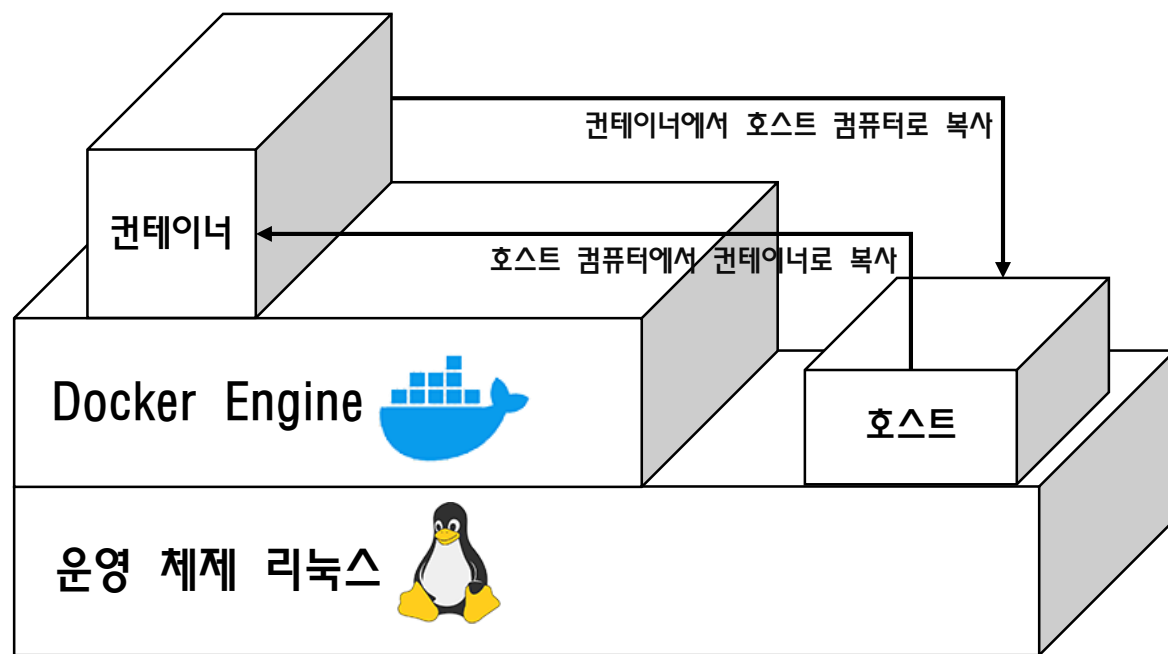


실전 컨테이너 사용법

컨테이너와 호스트 컴퓨터 간의 파일 복사

컨테이너와 호스트 컴퓨터 간의 파일 복사 방법에 대하여 살펴 봅니다.

파일의 복사는 컨테이너→호스트 컴퓨터, 호스트 컴퓨터→컨테이너로 양방향 복사가 가능합니다.



호스트 컴퓨터(host computer)

특정 네트워크 또는 시스템에서 다른 장치나 클라이언트를 수용하거나 관리하는 역할을 하는 컴퓨터를 의미합니다.

이 용어는 사용되는 맥락에 따라 약간 다른 의미로 쓰일 수 있으나, 대략 다음과 같은 의미를 가지고 있습니다.

1. 서버 역할로서의 호스트 컴퓨터

클라이언트-서버 모델에서, 서버로 작동하며 클라이언트 요청을 처리합니다.

2. 가상화에서의 호스트

가상 머신(VM)을 실행하기 위한 물리적인 컴퓨터를 호스트라고 합니다.

명령어(docker `cp)	설명
<code>docker cp 호스트경로 컨테이너이름:컨테이너경로</code>	호스트 컴퓨터에서 컨테이너로 파일을 복사하는 명령어입니다.
<code>docker cp 컨테이너이름:컨테이너경로 호스트경로</code>	컨테이너에서 호스트 컴퓨터로 파일을 복사하는 명령어입니다.

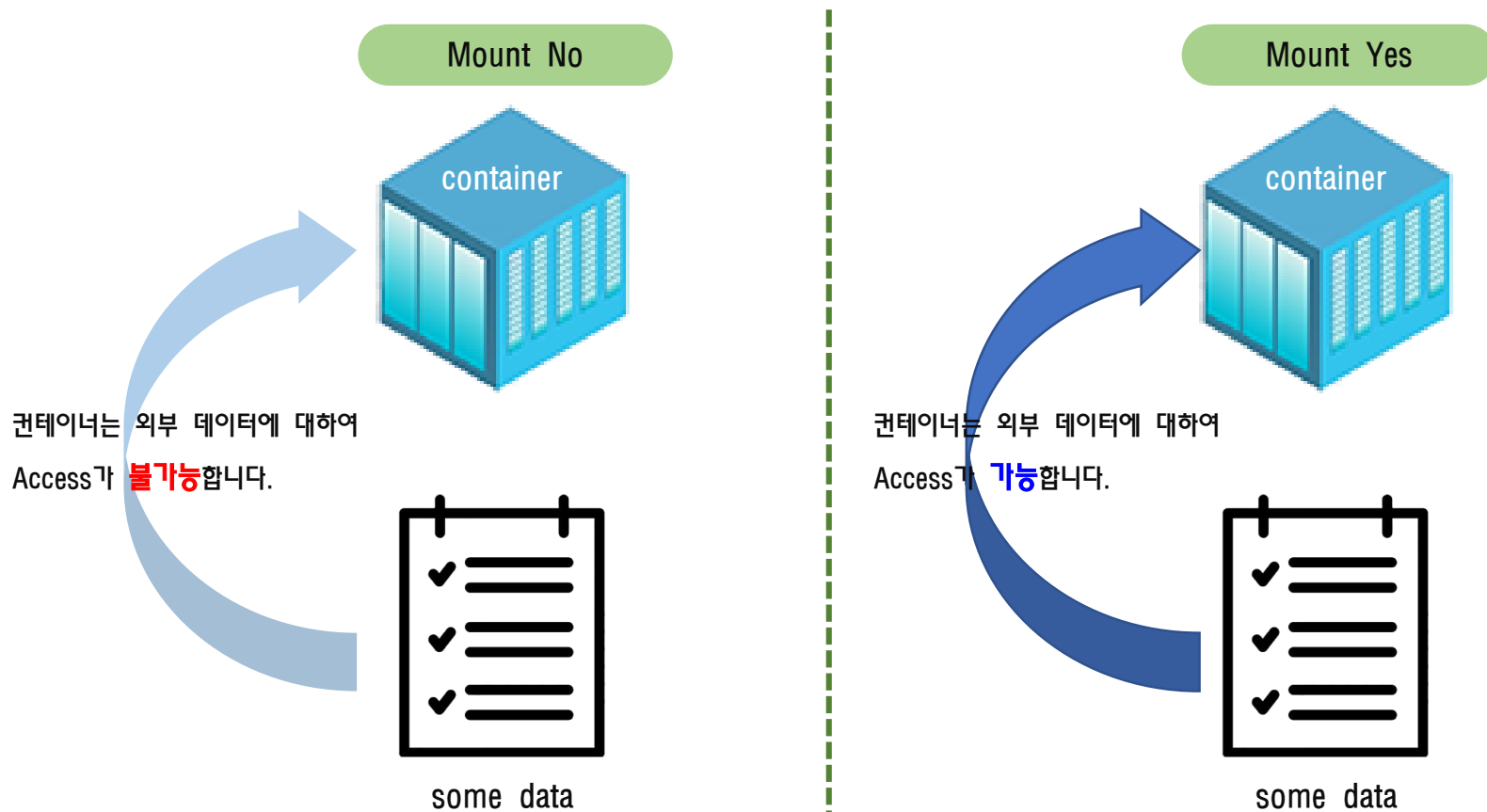
실전 컨테이너 사용법

'볼륨'(Volume) 마운트

볼륨은 저장소(storage)의 일부 영역을 분할시켜 놓은 영역을 의미합니다.

Mount는 파일 시스템 구조 내에 있는 일련의 파일들을 사용자나 사용자 그룹들이 이용할 수 있도록 만드는 과정을 의미합니다.

즉, Docker에서는 운영 체제 내의 특정 폴더(디렉토리) 또는 도커 엔진 내의 Volume를 컨테이너에 Disk 형태로 만들어서 사용하는 과정을 의미합니다.



≡ 실전 컨테이너 사용법

≡ 볼륨(Volume) 마운트

볼륨은 저장소(storage)의 일부 영역을 분할시켜 놓은 영역을 의미합니다.

Volume 관련 명령어

명령어	설명
<code>docker volume create</code>	새로운 볼륨 생성
<code>docker volume ls</code>	생성된 모든 볼륨 목록 조회
<code>docker volume inspect</code>	특정 볼륨의 상세 정보 확인
<code>docker volume rm</code>	특정 볼륨 삭제
<code>docker volume prune</code>	사용하지 않는 볼륨 모두 삭제
<code>docker volume help</code>	volume 관련 도움말 출력

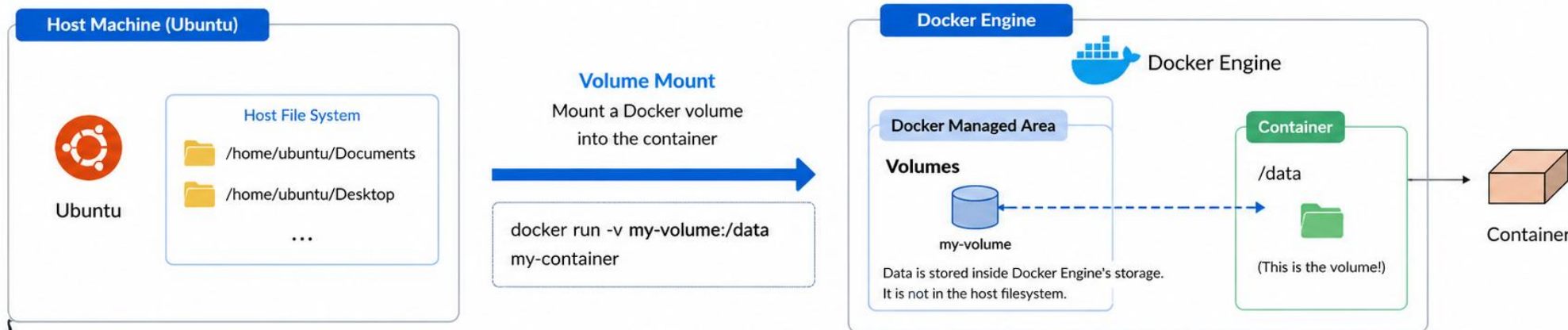
실전 컨테이너 사용법

스토리지 마운트

볼륨 마운트에 대하여 살펴 보겠습니다.

What is a Volume Mount?

- A volume mount mounts a volume created and **managed by Docker Engine** into a container.
- Volumes are stored in Docker's **managed area**, not in the host filesystem.



볼륨 마운트는 도커 엔진이 관리하는 영역 내에 만들어진 볼륨을 컨테이너 디스크 형태로 마운트합니다. 일반적으로 자주 사용하지는 않지만 삭제하면 안되는 파일을 저장하는 목적으로 많이 사용됩니다.

Key Points

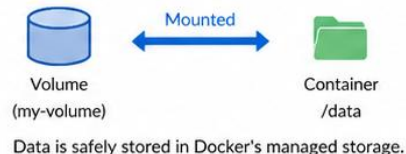
- Volumes are created and managed by Docker Engine. (Located in `/var/lib/docker/volumes`)
- Data in a volume persists even if the container is deleted.
- Useful for storing important data that should not be lost.

When to Use

Recommended for data that:

- ✓ Must be persisted
- ✓ Should not be tied to the host filesystem
- ✓ Needs to be shared between containers

Example



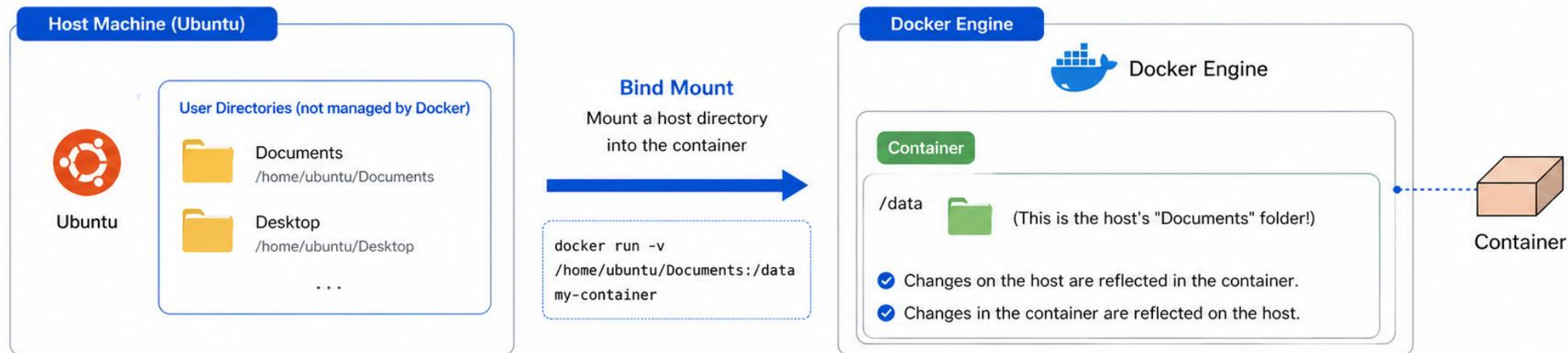
실전 컨테이너 사용법

스토리지 마운트

‘바인드 마운트’(bind mount)는 운영 체제의 일부 공간(디렉토리)을 컨테이너에 마운트 하는 방식입니다.

What is a Bind Mount?

- A bind mount mounts a directory (or file) **from the host operating system** into a container.
- It allows a container to directly access files and folders that are **outside of Docker's managed storage**.



Key Points



Bind mount uses existing directories on the host.



The host and container share the same data in real time.



Useful for development, logs, configuration files, and more.

Example



도커가 설치된 컴퓨터 내의 ‘문서 폴더’ 또는 ‘바탕화면 폴더’ 등 도커 엔진에서 관리하지 않는 영역의 기존 디렉토리를 컨테이너에 마운트하는 방식입니다.

≡ 실전 컨테이너 사용법

≡ 마운트 비교해 보기

2개의 마운트에 대하여 비교해 봅니다.

마운트 비교

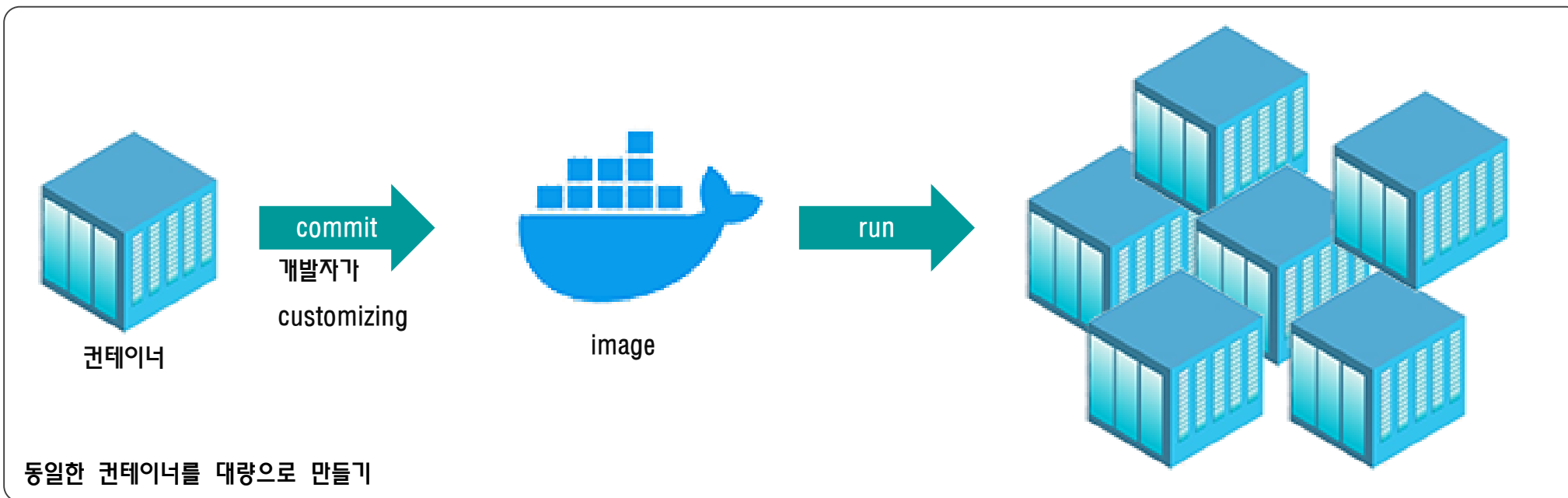
항목	볼륨 마운트	바인드 마운트
스토리지 영역	볼륨	디렉토리 또는 파일
물리적 위치	도커 엔진의 관리 영역	어디든지 가능
관리 방식	docker가 관리	사용자가 관리
이식성	플랫폼 독립적	플랫폼 의존적
마운트 절차	볼륨을 생성한 후 마운트	기본 파일이나 폴더를 마운트
내용 편집	도커 컨테이너를 통해서 편집 가능	일반적인 파일과 동일함
사용 사례	데이터 베이스, 로그 저장 등	개발 환경에서 소스 코드 공유
백업	절차가 복잡함	일반적인 파일과 동일함

≡ 실전 컨테이너 사용법

≡ 컨테이너로 이미지 만들기

컨테이너를 사용하여 이미지를 생성하는 법에 대하여 살펴 보겠습니다.

커스텀이징된 나만의 이미지가 존재하면, 동일한 구성의 컨테이너를 대량으로 만들수 있습니다.
또한, 다른 컴퓨터나 서버로 이동시키는 것도 가능합니다.



이미지로 변환 방법

방법 01 : commit 명령어로 컨테이너를 이미지로 변환할 수 있습니다.

방법 02 : Dockerfile 스크립트로 컨테이너를 이미지로 변환할 수 있습니다.

≡ 실전 컨테이너 사용법

≡ 컨테이너로 이미지 만들기

`commit` 커맨드로 컨테이너를 이미지로 변환하는 방법에 대하여 살펴 보겠습니다.

commit 명령어를 사용하면 현재 컨테이너의 상태를 그대로 이미지로 저장할 수 있습니다.

이 과정은 컨테이너에서 이루어진 변경 사항을 새로운 Docker 이미지로 저장할 때 매우 유용하게 사용할 수 있습니다.

docker 명령어

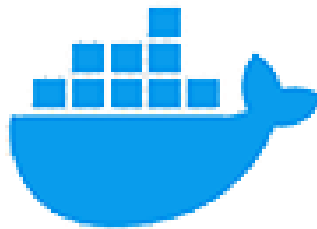
```
docker commit [OPTIONS] <container_id> <new_image_name>
```

rem 컨테이너를 사용하여 새로운 이미지를 생성합니다.

```
docker commit apache01 hohoho_apache
```



컨테이너



image

commit 명령어로 컨테이너를 이미지로 변환하기

commit을 하면?

파일 수정
설정 변경
프로그램 설치

≡ 실전 컨테이너 사용법

≡ 컨테이너로 이미지 만들기

`Dockerfile` 스크립트 파일로 이미지를 생성하는 명령어는 **build 옵션**을 사용합니다.

build 옵션

여기서 `-t` 옵션은 '이미지이름[:태그]' 형식으로 만들어 주는데, "태그"를 명시하지 않으면 기본 값으로 'latest'라는 단어가 붙습니다.

옵션	긴 옵션 이름	설명	사용 예시
<code>-t</code>	<code>--tag</code>	이미지 이름 및 태그 지정	<code>docker build -t myapp:1.0 .</code>
<code>-f</code>	<code>--file</code>	Dockerfile 경로 또는 파일명 지정	<code>docker build -f Dockerfile.dev -t myapp .</code>
	<code>--build-arg</code>	Dockerfile의 ARG 변수 전달	<code>docker build --build-arg VERSION=1.0 .</code>
	<code>--no-cache</code>	캐시 사용 안 하고 완전 새로 빌드	<code>docker build --no-cache -t myapp .</code>
	<code>--pull</code>	FROM 이미지 최신 버전 강제 pull	<code>docker build --pull -t myapp .</code>
	<code>--platform</code>	빌드 플랫폼 지정 (arm64, amd64 등)	<code>docker build --platform linux/amd64 -t myapp .</code>
	<code>--target</code>	멀티 스테이지 빌드의 특정 단계만 빌드	<code>docker build --target build -t myapp .</code>

≡ 실전 컨테이너 사용법

≡ 컨테이너로 이미지 만들기

`Dockerfile` 스크립트로 이미지를 생성하는 방법에 대하여 살펴 보겠습니다.

Dockerfile 스크립트 파일은 컴퓨터의 이미지 정보가 들어 있는 폴더 내에 위치하도록 합니다.

이외의 컨테이너에 넣은 파일이 있으면 동일한 폴더에 두도록 합니다.

docker 명령어

```
docker build -t created_image path
```

rem build 커맨드로 d:\docker\windows의 내용을 build_image라는 이미지로 빌드합니다.

```
docker build -t build_image d:\docker\windows
```

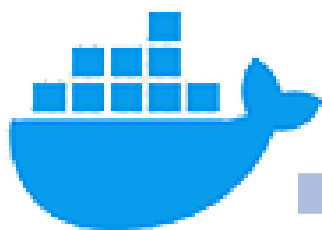


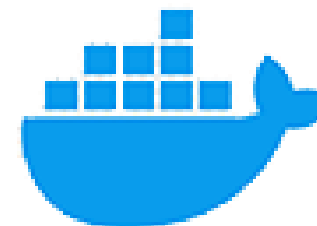
image 토대



docker file



컨테이너에
담을 추가 내용



생성된 image

Dockerfile 스크립트로 이미지 생성하기

실전 컨테이너 사용법

컨테이너로 이미지 만들기

Dockerfile 스크립트의 명령어(Instruction)에 대하여 살펴 보겠습니다.

주요 명령어(Instruction)는 컨테이너 이미지를 빌드하기 위해 제공되며, 각 명령어는 특정 작업을 수행하도록 설계되었습니다.

다음은 Dockerfile에서 자주 사용하는 명령어와 그 역할에 대한 설명입니다.

인스트럭션	설명
FROM	토대가 되는 이미지를 기반으로 새로운 이미지를 생성할 때 사용합니다. 항상 첫 번째로 작성되어야 합니다.
RUN	명령어를 실행하고 결과를 이미지에 포함시킵니다. 주로 패키지 설치나 의존성 설치에 사용됩니다.
CMD	컨테이너가 실행될 때 기본적으로 실행할 명령을 설정합니다. 단 하나만 사용할 수 있습니다. 여러 번 사용하면 마지막 CMD가 적용됩니다.
ENTRYPOINT	컨테이너가 실행될 때 실행할 명령을 설정하며, CMD와 함께 사용해 매개 변수를 전달할 수 있습니다.
WORKDIR	작업 디렉터리를 설정합니다. 이후 명령에서 기본 디렉터리로 사용됩니다.
COPY	호스트 시스템에서 파일이나 디렉터를 컨테이너 이미지에 복사합니다.
ADD	COPY와 유사하지만, 원격 URL에서 파일을 가져오거나 압축 파일을 자동으로 압축 해제할 수 있습니다.
ENV	환경 변수를 설정합니다.
EXPOSE	컨테이너에서 사용할 네트워크 포트를 정의합니다. 실제 포트는 `docker run`에서 지정해야 합니다.
ARG	빌드 타임에 전달할 변수(인자)를 정의합니다.(argument)
VOLUME	호스트와 컨테이너 간에 데이터를 공유하기 위한 볼륨을 지정합니다.

≡ 실전 컨테이너 사용법

≡ 컨테이너로 이미지 만들기

Dockerfile 스크립트의 인스트럭션에 대하여 살펴 보겠습니다.

주요 명령어(Instruction)는 컨테이너 이미지를 빌드하기 위해 제공되며, 각 명령어는 특정 작업을 수행하도록 설계되었습니다.

다음은 Dockerfile에서 자주 사용하는 명령어와 그 역할에 대한 설명입니다.

인스트럭션	설명
LABEL	이미지에 메타 데이터(키-값 쌍)를 추가합니다.
USER	컨테이너에서 실행할 사용자 계정을 설정합니다.
ONBUILD	부모 이미지를 기반으로 한 하위 Dockerfile에서 실행될 명령을 정의합니다.
HEALTHCHECK	컨테이너 상태를 주기적으로 확인하는 명령을 정의합니다.
SHELL	기본 셸을 변경합니다.

실전 컨테이너 사용법

컨테이너 내부에서 명령어를 실행하기

실행 중인 Docker 컨테이너 내부에서 명령어를 실행하는 방법에 대하여 살펴 봅니다.

"docker `exec`" 명령어는 실행 중인 Docker 컨테이너 내부에서 명령어를 실행할 때 사용됩니다.

이는 컨테이너에 새로운 프로세스를 시작하여 특정 작업을 수행하거나 디버깅 목적으로 활용할 수 있습니다.

컨테이너 내부 실행 관련 명령어

```
docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
```

관련 옵션

옵션	설명
-i(interactive)	표준 입력(입력 스트림)을 활성화합니다. 컨테이너 내에서 입력을 받을 수 있게 해줍니다.
-t(tty)	터미널 환경을 에뮬레이션하여 컨테이너와 상호 작용할 수 있게 해줍니다.
--detach 또는 -d	백그라운드에서 명령을 실행합니다.
--privileged	권한이 필요한 명령어를 실행할 수 있도록 컨테이너에 특권 모드를 부여합니다.
--user 또는 -u	명령어를 실행할 사용자 지정(`UID` 또는 사용자 이름).

사용 예시	설명
docker exec -it my_container /bin/bash	`my_container`라는 컨테이너에서 Bash 셸을 실행합니다.
docker exec my_container ls -l	컨테이너 내부에서 간단한 명령을 실행하고 결과를 바로 출력합니다. `ls -l` 명령이 컨테이너 내에서 실행되고 그 결과가 호스트 머신에 출력됩니다.
docker exec -u 1000 -it my_container whoami	컨테이너에서 UID가 1000인 사용자로 `whoami` 명령을 실행합니다.
docker exec -it -e SAMPLE=value my_container printenv SAMPLE	`-e` 옵션으로 환경 변수를 지정하고 해당 값을 확인할 수 있습니다.

≡ 실전 컨테이너 사용법

≡ 컨테이너 내부에서 명령어를 실행하기

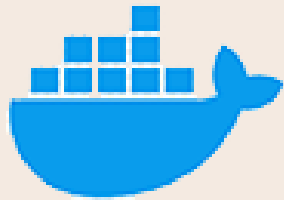
실행 중인 Docker 컨테이너 내부에서 명령어를 실행하는 방법에 대하여 살펴 봅니다.

활용 사례

활용 사례	설명
디버깅 및 점검	컨테이너 내부에서 문제를 디버깅하거나 로그를 점검할 때 유용합니다.
파일 시스템 확인	<code>`docker exec -it my_container ls /path`</code> 를 사용하여 컨테이너 파일 시스템 구조를 탐색할 수 있습니다.
임시 패치	컨테이너가 실행 중일 때 특정 파일을 수정하거나 일시적으로 환경 설정을 변경할 수 있습니다.
데이터베이스 접근	<code>docker exec -it mysql_container mysql -u root -p</code> 예시) MySQL 컨테이너에 접속하여 DB 명령 실행

Docker&Kubernetes

도커 컴포즈



≡ 매니 페스트 파일

도커 컴포즈에 무엇을 어떻게 구성할지를 설명해 놓은 문서를 의미합니다.

매니 페스트

어떤 컨테이너를 만들고

어떤 이미지에서 가져오고

어떤 포트로 연결하고

어떤 볼륨을 마운트하고

어떤 네트워크에 연결하고

매니페스트 파일

- MySql Container
- WordPress Container
- docker_network
- docker_volume

☰ 도커 컴포즈

여러 개의 Docker 컨테이너를 정의하고 관리하기 위한 도구입니다.

docker와 관련한 여러 개의 명령어를 한번에 실행/종료/폐기 등의 작업을 수행할 수 있도록 해주는 도구입니다.

시스템 구축과 관련된 명령어의 하나의 파일에 기재하여 복잡한 다중 컨테이너 애플리케이션을 설정, 실행, 및 관리할 때 유용합니다.

docker container run

MySQL Container

docker container run

WordPress Container

nwtwork create

docker_network

volume create

docker_volume



매니페스트 파일

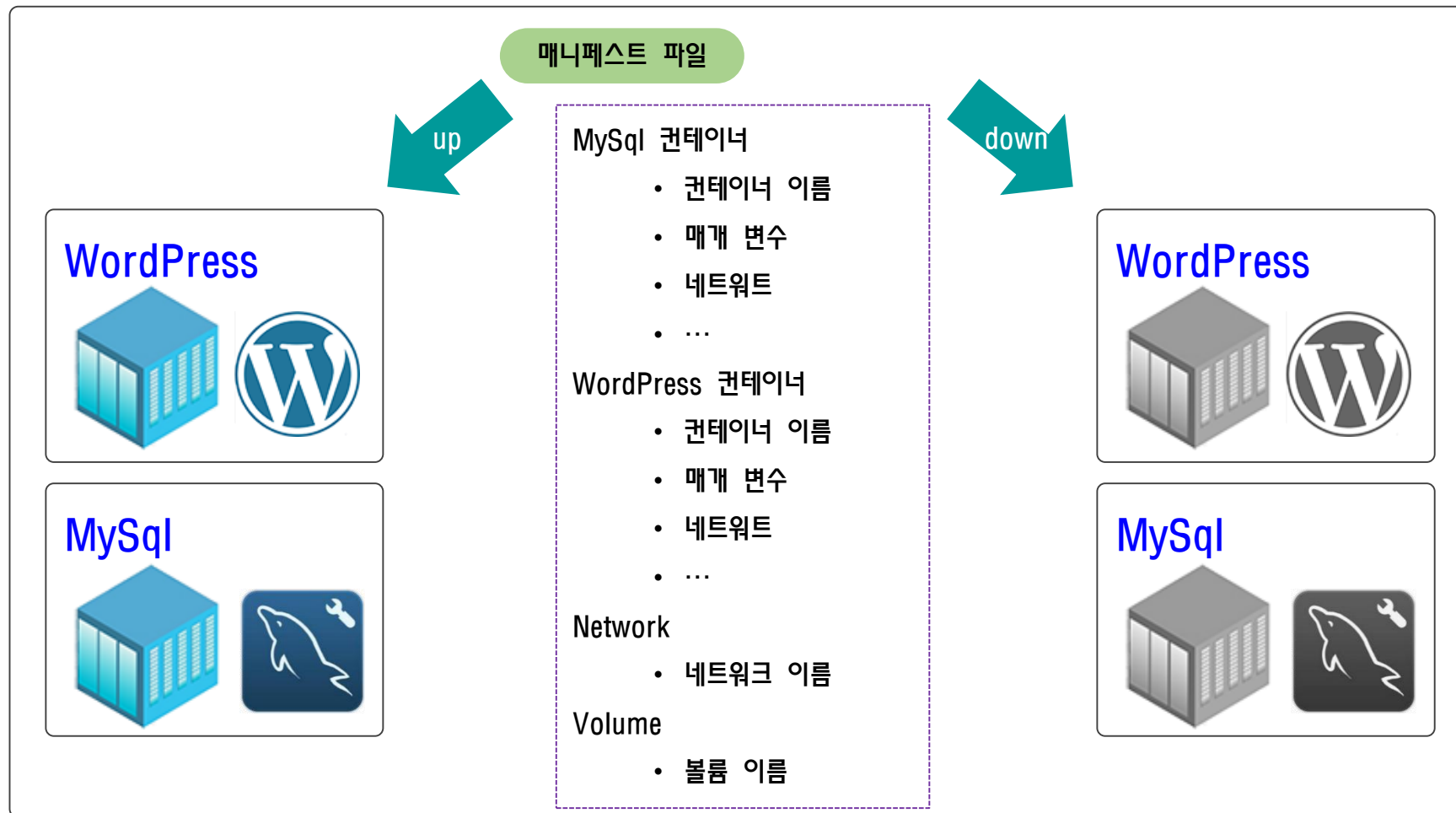
- MySQL Container
- WordPress Container
- docker_network
- docker_volume

도커 컴포즈를 사용하여 매니페스트 파일을 이용하여 여러 개의 명령어를 합쳐서 실행할 수 있습니다.

도커 `컴포즈`(`compose`)의 구조

시스템 구축에 필요한 설정 정보는 yaml 형식으로 작성합니다.

설정 파일에는 컨테이너나 볼륨을 "어떠한 방식으로 설정할 지"에 대한 정보들을 저장하고 있습니다.



도커 컴포즈(compose)의 주요 명령어

도커 컴포즈에서 사용되는 주요한 명령어는 다음과 같습니다.

주요 명령어

다음과 같은 명령어들 중에서, up, down, stop 정도만 기억하도록 합니다.

항목	설명
docker-compose up	정의된 컨테이너와 주변 환경 설정 정보를 생성해 줍니다.
docker-compose down	모든 서비스와 네트워크를 중지하고, 삭제합니다.
docker-compose ps	실행 중인 서비스(컨테이너) 목록을 확인합니다..
docker-compose logs	컨테이너가 출력한 내용을 화면에 보여 줍니다.
docker-compose exec <서비스 이름> <명령>	실행 중인 컨테이너에 명령어 전달.
docker-compose config	컴포즈 파일을 확인하고 내용을 출력합니다.
docker-compose port	포트 설정 내용을 출력합니다.
docker-compose start	컨테이너를 시작합니다.
docker-compose stop	컨테이너를 종료합니다.
docker-compose kill	컨테이너를 강제 종료합니다.
docker-compose run	컨테이너를 실행합니다.

도커 컴포즈의 주요 명령어

up 명령어에 대한 주요 옵션에 대하여 살펴 보도록 합니다.

주요 명령어

옵션	내용
-d	컨테이너를 백그라운드에서 실행합니다.
-p	생성되는 컨테이너의 이름을 직접 지정합니다. 명시하지 않으면 yml 파일이 들어 있는 디렉토리 이름을 컨테이너 이름으로 사용합니다.
-f	파일 이름을 구체적으로 명시하고자 할때 사용합니다.
--build	서비스 이미지를 실행 전에 강제로 다시 빌드합니다.
--no-build	이미지를 빌드하지 않고 기존 이미지를 사용합니다.
--force-recreate	기존 컨테이너를 삭제하고 새로 생성합니다.
--no-recreate	이미 존재하는 컨테이너는 재생성하지 않고 그대로 사용합니다.
--remove-orphans	docker-compose.yml에 정의되지 않은 서비스를 자동으로 중지하고 삭제합니다.
--scale SERVICE=NUM	특정 서비스의 컨테이너 개수를 조정합니다.

≡ 도커 컴포즈의 주요 명령어

down 명령어에 대한 주요 옵션에 대하여 살펴 보도록 합니다.

주요 명령어

옵션	내용
--volumes	서비스를 중지할 때 생성된 익명 볼륨을 삭제합니다.
--remove-orphans	정의되지 않은(고아) 컨테이너를 삭제합니다.
--rmi	서비스를 중지할 때 관련 이미지를 삭제합니다.
--timeout	컨테이너를 종료할 때 기다리는 시간(초)을 설정합니다. 기본값은 10초입니다.

도커 컴포즈

도커 컴포즈와 dockerfile의 차이점에 대하여 살펴 봅니다.

Docker Compose와 Dockerfile은 모두 Docker 환경에서 사용되지만, 목적과 역할이 다릅니다.
다음은 주요 차이점입니다.

특징	dockerfile	docker compose
목적	단일 컨테이너 이미지 정의	여러 컨테이너를 조합하여 애플리케이션 구성
파일 형식	Dockerfile(텍스트 형식)	YAML(docker-compose.yml)
사용 대상	컨테이너 이미지 생성	서비스 간 관계 설정 및 실행
주요 명령어	FROM, RUN, CMD, COPY, EXPOSE 등	services, build, ports, volumes, networks
기능	애플리케이션의 실행 환경 정의	컨테이너 오케스트레이션
단독 실행 가능 여부	가능 (단일 컨테이너 실행에 적합)	불가능 (단독으로 컨테이너 빌드 및 실행 불가)

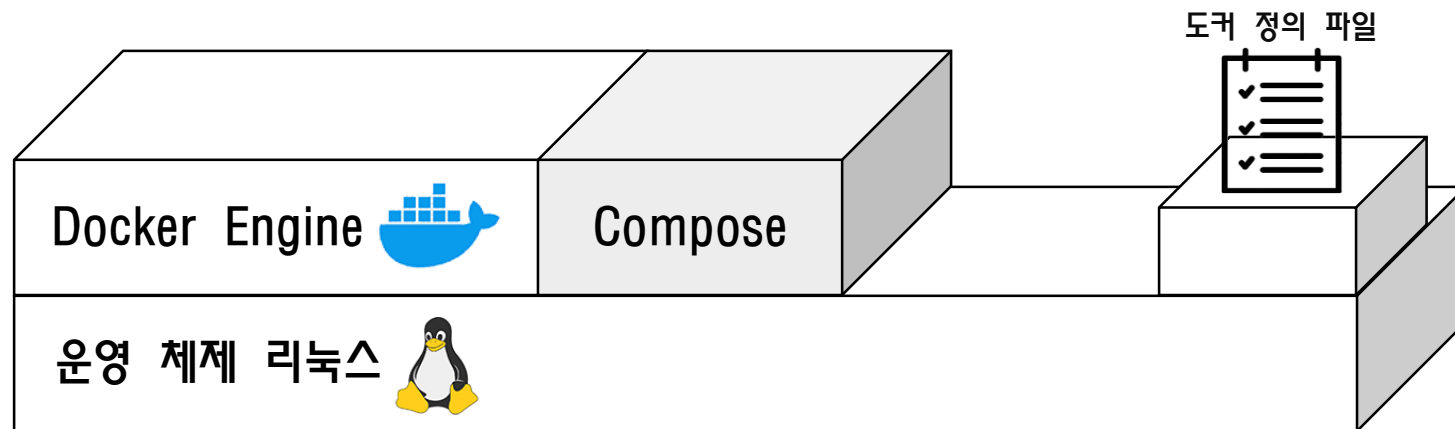
도커 컴포즈

도커 컴포즈의 사용법에 대하여 살펴 봅니다.

도커 컴포즈를 사용하려면 운영 체제 컴퓨터에 폴더를 하나 만들고, 해당 폴더에 도커 컴포즈 정의 파일 yaml을 배치합니다.

정의 파일의 이름은 미리 정해진 docker-compose.yml 이라는 이름을 사용해야 합니다.

파일에 정의된 내용이 도커 엔진에 전달이 되며, 생성된 컨테이너도 동일하게 도커 엔진 위에서 문제없이 잘 동작합니다.



☰ 도커 컴포즈 다뤄 보기

☰ 도커 컴포즈 다뤄 보기

도커 컴포즈 파일을 직접 작성해 보도록 합니다.

컴포즈 파일의 확장자(타입)는 yaml 또는 yml이고, 텍스트 형식이므로 사용하기 편한 텍스트 편집기를 사용하면 됩니다.

-f 옵션을 사용하면 임의의 파일 이름으로 작성이 가능하지만, 명시하지 않으면 "docker-compose.yml"으로 작성해야 합니다.

컴포즈 파일 작성 예시

```
version: "3" # 최신 버전에서는 사용하지 않음
services: # 컨테이너 관련 정보
networks: # 네트워크 관련 정보
volumes: # 볼륨 관련 정보
```

서비스(service)

도커 컴포즈와 쿠버네티스에는 컨테이너의 집합체(묶음)를 서비스라고 부릅니다.

컴포즈 파일 작성 규칙

이름 뒤에는 반드시 콜론 기호 : 를 붙여야 하며, 콜론 뒤에는 반드시 공백이 있어야 합니다.

문서 작성시 들여 쓰기는 탭을 사용하는 대신, 항상 동일한 개수의 띄워 쓰기 형식을 준수해야 합니다.

services, networks 등의 주항목 아래에는 이름을 작성합니다.

컨테이너의 설정 정보가 여러 개이면 줄을 바꿈한 다음, 하이픈 기호로 시작하여 들여 쓰기를 맞춰 줍니다.

기호는 주석으로 인식합니다.

도커 컴포즈

도커 컴포즈 파일에 작성할 항목들을 살펴 봅니다.

컴포즈에 작성할 항목들에 대하여 정리해 봅니다.

참고로 주 항목의 이름은 복수형이어야 함에 주의하도록 합니다.

주요 항목

항목	설명
services	컨테이너를 정의하는 영역입니다.
networks	네트워크를 정의하는 영역입니다.
volumes	볼륨을 정의하는 영역입니다.

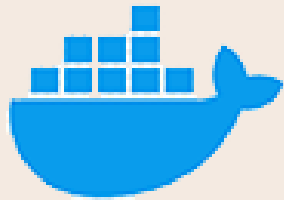
restart 설정 값

항목	설명
no	재시작하지 않습니다.
always	어떤 이유로 컨테이너가 멈추면 항상 재시작시킵니다.
on-failure	프로세스가 0 외의 상태로 종료되었다면 재시작합니다.
unless-stopped	종료시 재시작하지 않음. 그 외에는 재시작함

항목	docker run 커맨드의 옵션/매개변수	내용
image	이미지 매개 변수	사용할 이미지를 지정
networks	--net	접속할 네트워크를 지정
volumes	-v, --mount	스토리지 마운트를 설정합니다.
ports	-p	포트 설정
environment:	-e	환경 변수 설정
depends_on	없음	다른 서비스에 대한 의존 관계를 설정
restart	없음	컨테이너 종료시 재시작 여부를 지정

Docker&Kubernetes

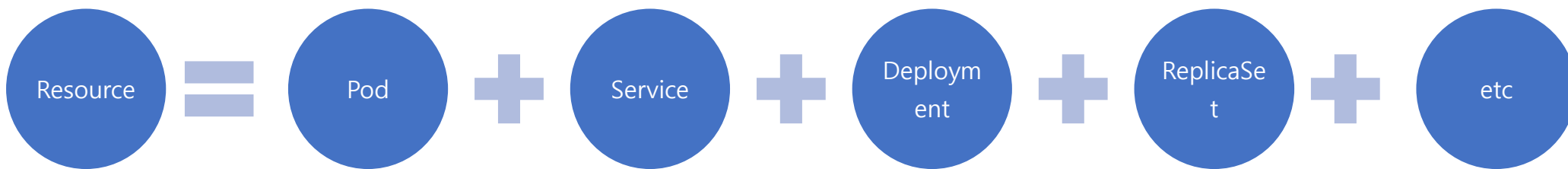
쿠버네티스



쿠버네티스 구성 요소

쿠버네티스와 관련된 구성 요소들은 다음과 같은 항목들이 있습니다.

관련 요소



구성 요소	역할
리소스(Resource)	쿠버네티스가 선언(YAML)으로 관리하는 모든 객체(Object)
파드(Pod)	컨테이너 1개 또는 그룹들이 들어 있는 최소 단위 한 개 이상의 컨테이너와 볼륨을 그룹화 시킨 객체 동일한 네트워크와 스토리지를 공유함
Deployment	Pod 자동 관리(재시작, 업데이트) Pod와 ReplicaSet의 관리 및 애플리케이션 자동 배포화
Service	Pod에 대한 안정적인 네트워크 엔드 포인트 제공 및 로드 밸런싱 역할
Ingress	도메인 기반 라우팅(nginx 역할)
Node	쿠버네티스가 설치되어 있는 서버
Cluster	여러 Node의 집합

쿠버네티스 구성 요소

파드(Pod) 컨테이너를 실행하는 최소 단위로써, 실행을 담당하는 리소스입니다.

역할(Role)

실제로 프로그램이 실행되는 공간
하나 이상의 컨테이너 포함 가능
IP를 가짐 (하지만 임시적)

Pod



Spring Boot 컨테이너 (주 역할)
Log Agent 컨테이너 (보조 역할)

0

메인 애플리케이션 옆에서 보조 역할을 수행하는 경우 바람직합니다.

Pod



X

스프링 부트와 MySQL은 역할과 확장 방식이 서로 다르므로 각각 별도의 파드로 구성하는 것이 일반적입니다.

예시

Spring Boot 파드
MySQL 파드

≡ 쿠버네티스 구성 요소

서비스(Service)는 파드에 대한 고정 접속 창구 역할을 하며 **접속 담당자 리소스**입니다.

역할(Role)

파드 IP가 바뀌어도 항상 같은 주소 제공
내부/외부 접근 가능
로드 밸런싱

종류

ClusterIP(기본)
NodePort
LoadBalancer

≡ 쿠버네티스 구성 요소

레플리카셋(ReplicaSet)은 파드의 개수를 보장하는 파드 개수 관리 담당자 입니다.

역할(Role)

지정된 개수의 Pod를 항상 유지 시켜 애플리케이션의 가용성을 보장

"파드를 n개 유지하라"

Pod가 죽으면 자동으로 새로운 Pod를 생성해 줍니다.

특징

직접 만들 일 거의 없음

Deployment가 대신 생성

≡ 쿠버네티스 구성 요소

디플로이먼트(Deployment)는 파드를 운영하기 위한 "운영 총괄 관리자" 역할을 합니다.

역할(Role)

ReplicaSet 생성
파드 생성/삭제/교체
무중단 배포(Rolling Update)
롤백 가능

특징

실무에서 가장 많이 쓰는 리소스
"Pod를 직접 쓰지 말고 Deployment를 써라"

쿠버네티스 구성 요소

기타 주요 리소스는 다음과 같은 항목들이 있습니다.

기타 리소스

항목	설명	역할
ConfigMap	설정 정보 관리	DB 주소 환경 변수
Secret	민감 정보 관리	비밀번호 토큰 인증서
Ingress	외부 HTTP/HTTPS 진입 관문	도메인 기반 라우팅 LoadBalancer 앞단
Namespace	리소스 논리적 구역	개발 / 운영 분리

쿠버네티스 구성 요소

`metadata`는 쿠버네티스가 리소스를 식별 · 분류 · 관리하기 위해 사용하는 정보입니다.

사용 예시

```
metadata:
  name: my-pod
  namespace: default
  labels:
    app: web
    tier: frontend
  annotations:
    description: "웹 서버 파드"
```

핵심 요소

항목	설명
name	리소스 이름 (필수)
namespace	소속 네임스페이스
labels	리소스 분류용 태그(Selector에서 사용)
annotations	설명, 설정 정보 등 메타 데이터
uid	클러스터 내부 고유 ID (자동 생성)
creationTimestamp	생성 시각

쿠버네티스 설치

윈도우에서 쿠버네티스를 설치하는 방법입니다.

Kubernetes 탭에서 [Create Cluster] 버튼을 클릭합니다.
Docker Desktop 내부에 내장 Kubernetes 클러스터(mini cluster)를 생성해 줍니다.
다소 시간이 좀 걸릴수 있습니다.(5분 내외 정도)

작업	설명
Kubernetes 컴포넌트 다운로드	kubelet, API server, scheduler 등
docker-desktop 클러스터 생성	노드 1개짜리 단일 클러스터
kubectl 자동 설정	Windows PowerShell에서 바로 사용 가능
kubeconfig 자동 업데이트	kubectl이 "docker-desktop"을 사용함

쿠버네티스 설치

윈도우에서 쿠버네티스를 설치하는 방법입니다.

Create Kubernetes Cluster

Cluster Type

- ☒ Kubeadm
Create a single-node cluster with kubeadm.
Version: v1.34.1

- ☐ kind
Create a cluster containing one or more nodes with kind. Requires the [containerd](#)

Advanced Settings

- ☐ Show system containers (advanced)
Show Kubernetes internal containers when using Docker commands.

Cancel

Create

Kubernetes Cluster Installation

Installation takes a few minutes and requires an internet connection.

Cancel

Install

쿠버네티스 설치

윈도우에서 쿠버네티스를 설치하는 방법입니다.

The screenshot shows the Docker Desktop application window. The top bar is blue with the Docker logo, 'docker.desktop PERSONAL', a search bar, and various icons. The left sidebar contains navigation options: Ask Gordon (BETA), Containers, Images, Volumes, Kubernetes (selected), Builds, Models, MCP Toolkit (BETA), Docker Hub, Docker Scout, and Extensions. The main panel displays the 'Kubernetes' section with a 'namespace' dropdown set to 'default'. Below this, a 'Cluster' section shows a single cluster in an 'Active' state, managed by 'kubeadm' with 1 node and Kubernetes version v1.34.1. There are 'Stop' and 'Edit cluster' buttons. Below the cluster table, there are two sections: 'Deployments' and 'Pods'. Both sections indicate 'No deployments' and 'No pods' respectively, and provide instructions on how to create a deployment or run a pod using 'kubectl' commands. The bottom status bar shows 'Engine running', system resources (RAM 3.52 GB, CPU 21.07%, Disk 7.94 GB used), and an 'Update available' notification.

Kubernetes [Give feedback](#)

namespace: default

Cluster Stop Edit cluster

Cluster	Cluster type	Nodes	Kubernetes version
Active	kubeadm	1	v1.34.1

Deployments

No deployments

Run the following command in a terminal to create a deployment

```
kubectl create deployment my-app --image=nginx
```

Pods

No pods

Run the following command in a terminal to run a container in a pod

```
kubectl run nginx --image nginx
```

Engine running | | : RAM 3.52 GB CPU 21.07% Disk: 7.94 GB used (limit 1006.85 GB) >_ Update available

kubectl apply

YAML 파일로 작성된 Kubernetes 리소스를 생성 · 업데이트하는 명령어입니다.

패키지 목록

옵션	긴 옵션 이름	설명
-f	--filename	YAML 파일 또는 디렉토리 지정
	--recursive	폴더 내부 파일 재귀 처리
-n	--namespace	특정 네임스페이스 적용
	--dry-run	실제 적용 없이 테스트
	--server-side	서버 사이드 apply
	--force-conflicts	충돌 시 강제 적용
-l	--prune	YAML에 없는 리소스 삭제
	--selector	라벨 선택
	--validate	YAML 검증

kubectl get

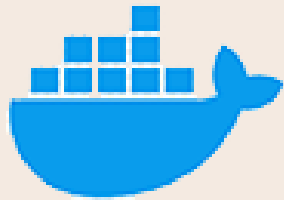
YAML 파일로 작성된 Kubernetes 리소스를 생성 · 업데이트하는 명령어입니다.

패키지 목록

옵션	긴 옵션 이름	설명
-f	--filename	YAML 파일 또는 디렉토리 지정
	--recursive	폴더 내부 파일 재귀 처리
-n	--namespace	특정 네임스페이스 적용
	--dry-run	실제 적용 없이 테스트
	--server-side	서버 사이드 apply
	--force-conflicts	충돌 시 강제 적용
-l	--prune	YAML에 없는 리소스 삭제
	--selector	라벨 선택
	--validate	YAML 검증

Docker&Kubernetes

리눅스에 도커 설치



리눅스에 도커 설치

리눅스에 도커 설치

리눅스에 도커 설치를 위하여 필요한 패키지 목록입니다.

패키지 목록

항목	설명
apt-transport-https	APT 패키지 관리자가 HTTPS 프로토콜 을 통해 패키지를 다운로드할 수 있도록 지원하는 패키지입니다. 기본적으로 APT는 HTTP, FTP 등 몇 가지 프로토콜을 지원하지만, HTTPS는 기본적으로 활성화되지 않았습니다.
ca-certificates	인증서 관리 를 위한 패키지 로, 주로 SSL/TLS 통신에서 신뢰할 수 있는 인증 기관(CA, Certificate Authority)의 루트 인증서를 포함하고 관리합니다.
curl	URL을 사용하여 데이터를 전송하거나 가져 오는 데 사용하는 명령줄 도구 및 라이브러리입니다.
software-properties-common	Ubuntu 및 다른 Debian 기반 Linux 배포판에서 소프트웨어 관리 와 관련된 유용한 도구들을 제공하는 패키지 입니다. 주로 PPA(Personal Package Archives)를 관리하거나 소프트웨어 소스를 편리하게 설정할 때 사용됩니다.
gnupg	디지털 서명 을 위한 패키지입니다.
lsb-release	리눅스 배포판 식별 을 위해 필요한 패키지 입니다.

GPG 키 등록

GPG는 GnuPG(GNU Privacy Guard)로 암호화 및 인증에 사용되는 도구입니다.

GPG(또는 GnuPG)란 GNU Privacy Guard의 줄임말로, 도커 이미지 인증을 확인할 때 사용하는 키를 의미합니다.

"<https://download.docker.com/linux/ubuntu/gpg>"는 Docker가 제공하는 GPG 키 파일의 URL입니다.

GPG 키 사용

다운로드/업로드 데이터가 "믿을 수 있는 사람에게서 왔는지" 검증
"전송 도중에 바뀌지 않았는지"등을 검증하고 보호

Docker 설치

다음은 Docker 설치 패키지 목록입니다.

패키지 목록

패키지 이름	설명	필수 여부
docker-ce (Community Edition)	컨테이너 실행 주체인 Docker 데몬으로써, Docker의 오픈 소스 무료 버전입니다. CE는 Community Edition이라는 뜻이에요.	필수
docker-ce-cli	docker 명령어를 제공하는 Command Line Interface) 도구입니다. docker, docker build, docker run, docker ps 같은 명령어 등이 있습니다.	필수
containerd.io	CNCF(Cloud Native Computing Foundation)가 관리하는 컨테이너 런타임(runtime) 엔진입니다. containerd.io는 리눅스 배포판에서 설치할 수 있도록 만든 패키지 이름입니다. 역할: 컨테이너 실행 (run), 컨테이너 이미지 다운로드, 저장 (pull, push), 컨테이너 생명 주기 관리 (시작, 중지, 삭제 등), 로우레벨 기능 담당 (네트워크, 파일 시스템 마운트 등)	필수
docker-buildx-plugin	고급 이미지 빌드 도구 (buildx)	옵션
docker-compose-plugin	여러 컨테이너를 한 번에 다루는 도구 (compose)	옵션

☰ Docker 설치

다음은 Docker 설치 패키지 각 항목들에 대한 내용입니다.

docker-ce (Community Edition)

정의:

- Docker의 핵심 엔진(daemon) 패키지입니다.

설명:

- Docker 데몬(dockerd)이 포함되어 있으며,
- Docker 컨테이너를 실제로 실행, 관리, 중지하는 중앙 서비스 역할을 합니다.

주요 기능:

- 컨테이너 실행/중단
- 이미지 생성 및 관리
- 네트워크, 볼륨 설정

필수 여부:

- 반드시 필요

docker-ce-cli (Command Line Interface)

정의:

- Docker 명령어 도구인 docker 명령을 제공하는 패키지입니다.

설명:

- 사용자 입장에서 명령어로 Docker와 상호작용할 수 있게 해줍니다.
- docker run, docker ps, docker build 등을 실행할 수 있게 함.

주요 기능:

- 사용자가 Docker에게 명령 전달
- 내부적으로는 Docker 데몬에게 API 요청을 보냄

필수 여부:

- 반드시 필요

Docker 설치

다음은 Docker 설치 패키지 각 항목들에 대한 내용입니다.

containerd.io

정의:

- Docker가 내부적으로 사용하는 컨테이너 실행 엔진

설명:

- Docker가 실제로 컨테이너를 실행할 때 내부적으로 containerd를 사용
- CNCF(Cloud Native Computing Foundation) 소속의 독립 프로젝트이기도 함
- Docker 없이도 containerd만 단독으로 사용할 수 있음

주요 기능:

- 이미지 가져오기/푸시
- 컨테이너 시작/중지
- 로우 레벨 자원 관리

필수 여부:

- Docker CE에서 자동 포함

docker-buildx-plugin

정의:

- 멀티플랫폼 이미지를 빌드할 수 있게 해주는 고급 빌드 도구

설명:

- 기존 docker build보다 더 많은 기능 지원 (예: ARM/AMD 이미지 동시에 빌드)
- 내부적으로 BuildKit 사용

주요 기능:

- 병렬 빌드, 캐시, 멀티 아키텍처 이미지 빌드 지원

필수 여부:

- 일반 사용자는 선택적
- 고급 이미지 빌드에 필요

☰ Docker 설치

다음은 Docker 설치 패키지 각 항목들에 대한 내용입니다.

docker-compose-plugin

정의:

- 여러 컨테이너를 한번에 구성할 수 있게 해주는 Compose 기능 플러그인

설명:

- 예전에는 docker-compose라는 별도 바이너리였지만,
- 이제는 Docker 명령어 안에 plugin으로 포함

주요 기능:

- YAML 파일을 기반으로 여러 컨테이너 한꺼번에 정의/실행
- 예: 웹 + DB 컨테이너를 함께 띄우기

필수 여부:

- 단일 컨테이너는 불필요
- 복잡한 서비스 구성 시 필요